



Cairo University

Faculty of Computers and Artificial Intelligence
Information Systems Department



Graduation Project Report



TourMate AI

Agentic AI Tourism Auto-Pilot System

Prepared by:

Abdelrahman Mostafa Sayed	20220197
Omar Mohamed El-Sayed	20221109
Mohamed Sayed Shebl	20220290
Donia Kareem Mohamed	20221051
Asmaa Yasser Hussien	20221022
Alaa Ibrahim Hosny	20221024

Under the supervision of:

Prof. Sherif Zahran

Eng. Dina Amr

*Department of Information Systems, Faculty of Computers and
Artificial Intelligence, Cairo University, Egypt*

July 2026

Abstract

The rapid growth of digital travel information has made trip planning a fragmented and cognitively demanding process, requiring travelers to coordinate across multiple platforms to create a single itinerary. This often leads to inefficient schedules, inconsistent travel experiences, and limited personalization. To address these challenges, this project presents **TourMate AI**, an Agentic AI Tourism Auto-Pilot System that autonomously generates, optimizes, and simulates personalized travel itineraries while preserving user control through a human-in-the-loop approach. TourMate AI employs an agent-based artificial intelligence architecture capable of reasoning about user preferences, time constraints, geographic distance, activity pacing, and contextual travel requirements. The platform supports multimodal interaction, enabling users to express their travel intentions through both natural language and images, thereby enhancing personalization. In addition, it maintains an evolving behavioral profile that continuously adapts recommendations based on user interactions and feedback. The developed system produces coherent, optimized, and explainable travel itineraries within a safe simulation environment, allowing users to review, modify, and approve plans before execution. Experimental evaluation demonstrates that TourMate AI effectively reduces the complexity of manual travel planning while delivering adaptive, transparent, and personalized itinerary recommendations. The proposed approach bridges the gap between traditional manual planning and static automated solutions, providing an intelligent and user-centered travel planning experience for modern travelers.

Table of Contents

CHAPTER (1) INTRODUCTION	1
1.1 Background and Motivation.....	1
1.2 Problem Statement.....	2
1.3 Objectives	3
1.4 Project Scope and Limitations	4
1.4.1 Scope.....	4
1.4.2 Limitations	5
1.5 Project Methodology.....	7
1.6 Project Report Outline.....	10
CHAPTER (2) MARKET AND LITERATURE SURVEY	11
2.1 Overview of Current Tourism Planning Apps	11
2.2 Related Work: Key Applications in the Market.....	11
2.2.1 Vacay (The Chatbot)	11
2.2.2 Wanderlog (The Digital Notebook).....	12
2.2.3 Roam Around (The Instant Planner).....	13
2.2.4 Guide Geek (The Text Message Helper)	13
2.2.5 Google Travel (The Big Data Apps).....	14
2.3 Comparative Analysis	15
2.4 Gaps in the Market	16
2.5 How TourMate AI Solves These Problems.....	16
2.6 Conclusion	17
CHAPTER (3) TOURMATE AI ANALYSIS	18
3.1 Functional Requirements.....	18
3.2 Non-Functional Requirements	22
3.3 User Personas.....	24
3.4 Use-Case Diagram	29
3.5 Use-Case Narratives.....	30
3.6 Class Diagram (view)	52
CHAPTER (4) TOURMATE AI DESIGN AND IMPLEMENTATION	53
4.1 System Architecture	53
4.2 Entity Relationship Diagram (view).....	54
4.3 Sequence Diagrams.....	55

4.4	Tools and Technologies Used	62
4.4.1	Core Development Frameworks	62
4.4.2	AI Orchestration and Vision.....	63
4.4.3	Data Management	64
4.4.4	External Services and Integration	65
4.4.5	Development and Design Tools	66
4.5	Application Screens (UI)	67
CHAPTER (5)	TOURMATE AI TESTING AND EVALUATION	73
5.1	Testing Environment	73
5.2	Test Cases.....	74
	Category 1: Authentication & Account Management	74
	Category 2: Intelligent Trip Profiling & Personalization.....	77
	Category 3: Conversational & Multimodal Trip Requests.....	79
	Category 4: AI-Powered Itinerary Generation & Editing	82
	Category 5: Place Discovery & Exploration.....	86
	Category 6: Flight & Hotel Booking.....	88
	Category 7: Payment Management	92
	Category 8: Reviews & Personal Bookmarks.....	95
	Category 9: Image-Based Travel Inspiration.....	99
	Category 10: Trip Lifecycle Management	100
	Category 11: Conversation Flow & Context Management.....	104
	Category 12: Transparency & Continuous Improvement	107
	Category 13: System Health & Monitoring.....	108
CHAPTER (6)	CONCLUSIONS AND FUTURE WORK.....	109
6.1	Conclusions	109
6.2	Lessons Learned	110
6.3	Future Work	111
REFERENCE		112

List of Figures

Figure 1.5.1 Hybrid waterfall and agile approach	9
Figure 1.5.2 High-level sprint timeline and epic progress – TourMate AI	9
Figure 3.3.1 First-Time Tourist Persona	24
Figure 3.3.2 Frequent Traveler Persona	25
Figure 3.3.3 Local Explorer Persona	26
Figure 3.3.4 Government & Tourism Authorities Persona.....	27
Figure 3.3.5 Private Tourism Agencies Persona	28
Figure 3.4.1 Use Case Diagram	29
Figure 3.6.1 Class Diagram	52
Figure 4.1.1 System Architecture	53
Figure 4.2.1 Entity Relationship Diagram	54
Figure 4.3.1 User Authentication Sequence Diagram.....	55
Figure 4.3.2 AI Chat & Trip Planning Sequence Diagram.....	56
Figure 4.3.3 AI Itinerary Modification Sequence Diagram.....	57
Figure 4.3.4 Flight Booking Sequence diagram.....	58
Figure 4.3.5 Hotel Booking Sequence diagram	59
Figure 4.3.6 Manage Payments Sequence diagram.....	60
Figure 4.3.7 Coordinated Cancellation & Refund Sequence Diagram.....	61

List of Tables

Table 2.3.1 Feature Comparison of AI Travel Planning Platforms.....	15
Table 3.5.1 Authenticate & Manage Account Scenario	30
Table 3.5.2 Manage Trips & Itineraries Scenario	32
Table 3.5.3 Chat with AI & Plan Trip Scenario.....	34
Table 3.5.4 Explore & Discover Places Scenario	37
Table 3.5.5 Book Flights Scenario	39
Table 3.5.6 Manage Payments Scenario.....	41
Table 3.5.7 Review Places & Give Feedback Scenario	44
Table 3.5.8 Save & Manage Places Scenario.....	46
Table 3.5.9 Upload & Analyze Images Scenario	48
Table 3.5.10 Monitor System Health Scenario.....	50
Table 5.1.1 Testing Types and Tools Table	73
Table 5.2.1 User Registration via Email Test Case.....	74
Table 5.2.2 User Registration with Existing Email Test Case	74
Table 5.2.3 User Login (Auto-Registration) Test Case.....	75
Table 5.2.4 Unauthenticated Access to Protected Endpoint Test Case	75
Table 5.2.5 Expired or Invalid Firebase Token Test Case	76
Table 5.2.6 Trip Profile Created After Status Update to Active Test Case.....	77
Table 5.2.7 Traveler Persona Updated After Trip Approval Test Case.....	77
Table 5.2.8 Preference-Based Place Reranking Test Case	78
Table 5.2.9 WebSocket New Chat Connection Test Case	79
Table 5.2.10 Full AI Planning Pipeline via WebSocket Test Case.....	79
Table 5.2.11 Image Upload in Chat Test Case.....	80
Table 5.2.12 LLM API Timeout (AI Pipeline Failure) Test Case	80
Table 5.2.13 Follow-up Itinerary Modification Test Case	82
Table 5.2.14 Itinerary Route Optimization Test Case	82
Table 5.2.15 Itinerary Validation Checks	83
Table 5.2.16 Edit Intent Classification Test Case.....	83
Table 5.2.17 LLM Malformed JSON (Itinerary Parsing Crash) Test Case	84
Table 5.2.18 Explore Places (Default to Egypt) Test Case.....	86
Table 5.2.19 Explore Places Filtered by Category Test Case	86
Table 5.2.20 Get Place Detail Test Case.....	87
Table 5.2.21 Semantic Search with Natural Language Test Case	87
Table 5.2.22 Smart Flight Search by City Names Test Case	88
Table 5.2.23 Initiate Flight Booking (Price + PaymentIntent) Test Case	88
Table 5.2.24 Confirm Flight Booking After Payment Test Case.....	89
Table 5.2.25 Cancel Flight Booking Test Case.....	89
Table 5.2.26 Hotel Selection with Preference Scoring Test Case	90
Table 5.2.27 Amadeus API Rate Limiting (Throttling) Test Case	90
Table 5.2.28 Initiate Stripe Payment for Booking Test Case	92

Table 5.2.29 Confirm Booking After Successful Payment Test Case	92
Table 5.2.30 Combined Flight & Hotel Booking Initiate Test Case	93
Table 5.2.31 Combined Booking Confirm After Payment Test Case	93
Table 5.2.32 Payment Failure (Booking Not Confirmed) Test Case	94
Table 5.2.33 Create Review for a Place Test Case	95
Table 5.2.34 Duplicate Review Returns Conflict Test Case	95
Table 5.2.35 Get Reviews for a Place with Summary Test Case	96
Table 5.2.36 Toggle Like on Review Test Case	96
Table 5.2.37 Save a Place Test Case	97
Table 5.2.38 Duplicate Save Returns Conflict Test Case	97
Table 5.2.39 List Saved Places with Full Details Test Case	98
Table 5.2.40 Image Upload Triggers Vision Feature Extraction Test Case	99
Table 5.2.41 Get Images for a Trip Test Case	99
Table 5.2.42 Create Trip with Minimal Fields Test Case	100
Table 5.2.43 Create Trip with All Fields Test Case	100
Table 5.2.44 List User Trips Test Case	101
Table 5.2.45 Get Single Trip Details Test Case	101
Table 5.2.46 Delete Trip and Cascaded Data Test Case	102
Table 5.2.47 Update Trip Status Test Case	102
Table 5.2.48 Cancel Trip with Bookings Test Case	103
Table 5.2.49 Clear Chat History Test Case	104
Table 5.2.50 Itinerary Card Data in Chat History Test Case	104
Table 5.2.51 State Recovery from DB Snapshot Test Case	105
Table 5.2.52 WebSocket Reconnection Preserves State Test Case	105
Table 5.2.53 Concurrent WebSocket Messages (Race Condition) Test Case	106
Table 5.2.54 Submit Trip Feedback Test Case	107
Table 5.2.55 Get All Feedback by Current User Test Case	107
Table 5.2.56 Concurrent WebSocket Message Processing Test Case	108
Table 5.2.57 WebSocket Chat with Invalid Token Test Case	108

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
VLM	Vision-Language Model
OAuth	Open Authorization Protocol
UI	User Interface
QA	Quality Assurance
UAT	User Acceptance Testing
NLP	Natural Language Processing

Chapter (1) Introduction

This chapter provides an overview of the **TourMate AI** project. It introduces the background and motivation behind the system, defines the problem statement, outlines the project objectives, clarifies the project scope and limitations, describes the adopted methodology, and presents the structure of the project report.

1.1 Background and Motivation

Tourism planning has become increasingly complex due to the large amount of available information, diverse traveler preferences, and dynamic constraints such as time, budget, and location. Travelers are often required to rely on multiple platforms, including booking websites, blogs, social media, and mapping services. This fragmentation frequently results in inefficient planning, poorly optimized itineraries, and unsatisfactory travel experiences.

Recent advances in Artificial Intelligence (AI), particularly in agentic systems, large language models, and vision-language models, have created new opportunities to improve the tourism planning process. Modern AI systems can understand user intent, learn from behavioral patterns, process visual inputs to infer preferences, and generate optimized recommendations dynamically.

TourMate AI is motivated by the need for an intelligent tourism assistant that goes beyond static and generic recommendations. The project aims to apply agentic AI concepts to autonomously plan, optimize, and simulate personalized travel itineraries while maintaining a human-in-the-loop approach. By doing so, TourMate AI seeks to simplify travel planning, reduce cognitive effort, and support better decision-making for travelers.

1.2 Problem Statement

Despite the wide availability of online travel platforms, existing solutions remain limited in their ability to autonomously manage personalized trip planning. Most travel planning processes are still manual and time-consuming, requiring users to gather information from multiple sources and construct itineraries themselves. Current recommendation systems are often generic and fail to adapt dynamically to evolving user preferences or behavior.

Furthermore, most platforms lack autonomous itinerary optimization and provide limited support for behavioral learning and image-based preference extraction. Recommendation processes are often opaque, making it difficult for users to understand or trust the generated plans. As a result, there is a clear gap between user expectations for intelligent, transparent, and adaptive travel planning and the capabilities of current tourism systems.

Therefore, there is a need for an intelligent tourism system that can autonomously generate, optimize, and adapt travel itineraries while preserving transparency and allowing meaningful user control.

1.3 Objectives

The primary objective of **TourMate AI** is to design and implement an intelligent, agentic tourism system that autonomously plans and optimizes personalized travel itineraries while maintaining user control over decision-making.

The specific objectives of the project are to:

- Develop a behavioral profiling mechanism that learns from user interactions and preferences to enable adaptive personalization.
- Automatically generate complete and feasible travel itineraries based on user intent, constraints, and inferred preferences.
- Optimize itineraries with respect to time efficiency, geographic coherence, pacing, and thematic consistency.
- Support multimodal input by incorporating both natural language and image-based data for enhanced preference extraction.
- Provide a clear and transparent itinerary presentation that allows users to review, modify, and approve plans before execution.
- Enable flexible itinerary modification through preference adjustment, targeted edits, and full regeneration, allowing users to refine plans without restarting the planning process.
- Simulate booking and execution processes in a controlled environment to enable evaluation without financial or legal risk.
- Ensure system transparency through logging and explainable decision flows to improve trust and usability.

1.4 Project Scope and Limitations

This project focuses on the design and development of an intelligent tourism planning and recommendation system implemented within a simulated environment. The primary emphasis is placed on modeling agentic AI behavior for user profiling, itinerary generation, optimization, and interactive decision-making, rather than on deploying a production-ready commercial travel platform. Real-world booking, payment, and execution processes are intentionally simulated to allow controlled experimentation and evaluation within academic constraints.

1.4.1 Scope

The scope of TourMate AI includes the following components:

- **Behavioral profiling and preference modeling:** The system analyzes user interactions, approvals, skips, and feedback to build and continuously refine personalized behavioral profiles.
- **Personalized itinerary generation:** The system autonomously generates multi-day travel itineraries tailored to user preferences, time constraints, and thematic interests.
- **Itinerary optimization:** Generated itineraries are optimized with respect to time allocation, travel distance, pacing, and thematic coherence to ensure feasible and balanced travel plans.
- **Tour stop organization:** Tour stops are organized within the itinerary according to category and suitability for different times of day, enabling flexible customization and adaptive travel planning.
- **Multimodal input support:** The system supports text-only, image-only, and combined text-and-image inputs to infer user interests and enhance personalization.
- **Structured itinerary presentation:** Itineraries are presented in a clear, day-by-day format with organized schedules and location breakdowns to improve usability and readability.

- **User interaction and control mechanisms:** Users can review, modify, approve, or cancel generated itineraries through interactive feedback loops.
- **Booking and execution simulation:** The system simulates the accommodation booking process, while flight search and reservation, along with payment processing, are integrated with real third-party services operating in test and sandbox environments, allowing full end-to-end evaluation without real financial transactions.
- **Flight search and reservation:** The system supports searching, comparing, and selecting flights based on user-specified travel dates, class preferences, and route options, integrated as part of the overall booking flow.
- **System logging and evaluation support:** Interaction logs and system outputs are recorded to support performance evaluation, transparency, and analytical assessment.

1.4.2 Limitations

The project is subject to the following limitations:

- **Booking and payment processes:** Accommodation booking is implemented in simulation mode, without integration with real-world hotel or accommodation providers. Flight search and reservation, as well as payment processing, are integrated with real third-party services operating in test or sandbox environments rather than live production systems, allowing realistic evaluation without real financial transactions.
- **Reliance on third-party data sources and APIs:** The system depends on external services for place information, images, location and routing data, flight information, and session-state management. These services may impose rate limits, require internet connectivity, change their policies over time, restrict advanced features behind paid subscriptions, or experience temporary downtime, which can affect data availability, accuracy, and system performance.

- **Accuracy of AI-based inference:** Although Vision-Language Models can extract meaningful features from images, the inferred preferences may not always accurately reflect the user's true interests. Image quality, ambiguity, or unrelated visual content may lead to incomplete or noisy feature extraction, which can impact personalization quality.
- **Cold-start problem for new users:** For new users with little or no interaction history, personalization initially depends on baseline default profiles. This is partially mitigated through conversational slot-filling, in which the system gathers destination, duration, and interest information through natural dialogue before generating an itinerary, along with a default profile fallback when no preferences have yet been expressed.
- **Time and resource limitations typical of academic environments:** As a result, the system focuses on core functionalities and does not include advanced features such as real-time booking integration, multi-modal transportation optimization, or comprehensive user feedback analysis.

1.5 Project Methodology

The TourMate AI project uses Hybrid Methodology. This approach combines the structure of the Waterfall model with the flexibility of Agile. This mix is chosen because the project needs clear planning for the system architecture but also needs flexible testing for the AI features.

TourMate AI is a complex system that includes AI agents, itinerary planning, and data integration. Some parts, like the core design, must be defined early (Waterfall). Other parts, like the AI logic, need to be tested and improved many times (Agile). The Hybrid approach allows the team to get the benefits of both methods.

Rationale for Choosing Hybrid Methodology:

- **Structured Foundation:** The project starts with clear steps to define the requirements and the system design. This ensures the project has a stable base before coding begins.
- **Flexible Development:** Once the foundation is ready, the team uses Agile "sprints" (short work cycles). This allows for experimenting with the AI and fixing issues quickly based on feedback.

Methodology in Action

As illustrated in Figure 1.1 below, the project follows a clear workflow:

1. Planning and Analysis (Waterfall Phase) Following the left side of the figure, the project begins with a structured **Planning** phase. During this time, the team focuses on:

- **Initial Requirements Analysis & Design:** Defining exactly what the system must do and how the architecture will work.
- **Project Backlog:** Creating a list of all tasks and features needed for the project.
- **Sprint Plan:** Deciding which features will be built in the upcoming Agile cycles.

2. Agile Sprints (Iterative Phase) The center of the process involves repeating **Agile Sprints**. As shown in the loop in Figure 1.1, each sprint goes through four key steps:

- **Design:** Planning the specific feature for that sprint.
- **Build:** Writing the code for the AI agents and optimization logic.
- **Test:** Performing unit testing to validate individual components and integration testing to ensure correct interaction between system modules.
- **Partial UAT:** Conducting "Partial User Acceptance Testing" to see if the new features meet user needs.
- **Note:** Daily meetings are held to keep the team aligned.

3. Finalization (Waterfall Phase) Once the iterations are complete, the project moves to the final stages shown on the right side of the figure:

- **QA (Quality Assurance):** Conducting system testing and final User Acceptance Testing (UAT) to validate the complete system, identify remaining defects, and ensure that all functional and non-functional requirements are satisfied.
- **Deployment:** The final version of TourMate AI is validated and ready for presentation.

Benefits of this Approach

- **Clarity:** Clear milestones help keep the project on track for academic deadlines.
- **Adaptability:** The team can improve the AI logic without changing the whole system.
- **Quality:** Testing happens often (in every sprint) and again at the end (QA), ensuring a robust final product.

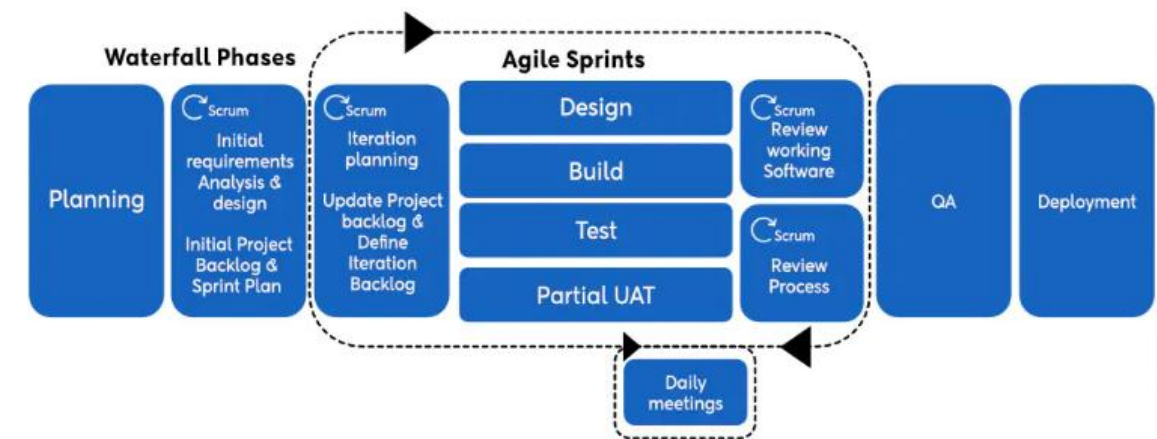


Figure 1.5.1 Hybrid waterfall and agile approach

The project followed an iterative Agile-based development lifecycle. Each sprint delivered a functional subsystem that incrementally expanded TourMate AI capabilities. Early sprints focused on infrastructure and authentication, while later sprints introduced AI reasoning, personalization, booking simulation, and continuous learning. This ensured progressive validation and risk mitigation throughout development.

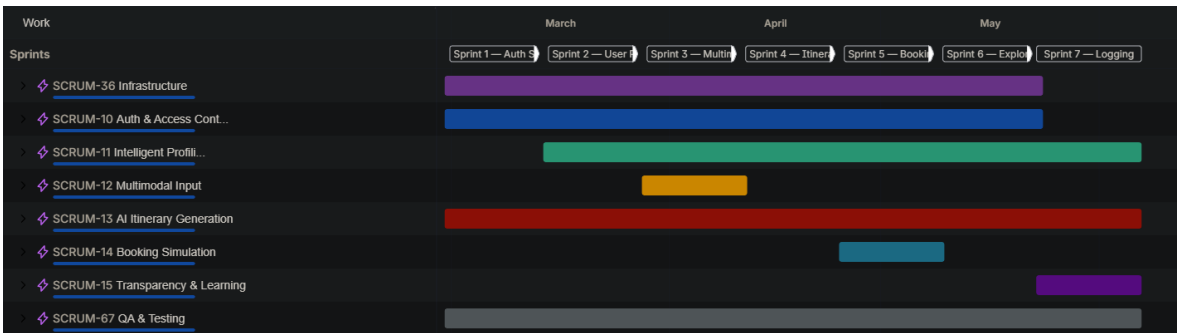


Figure 1.5.2 High-level sprint timeline and epic progress – TourMate AI

1.6 Project Report Outline

- **Chapter 2** presents the market and literature survey, analyzing existing tourism planning solutions and AI-powered travel recommendation systems, and identifying gaps that TourMate AI addresses.
- **Chapter 3** presents the system analysis, including functional and non-functional requirements, stakeholder definitions, user personas, system scenarios, and essential UML diagrams such as use case, class, sequence, and entity-relationship diagrams.
- **Chapter 4** presents the system design and implementation, describing the architecture, AI agent coordination, behavioral profiling mechanisms, itinerary generation and optimization algorithms, multimodal input processing, user interface design, prototype screens, and the technologies, tools, and frameworks used with selection justifications.
- **Chapter 5** presents the testing and evaluation of the system, including unit testing, integration testing, system testing, and user acceptance testing, as well as performance evaluation and validation of the generated travel itineraries.
- **Chapter 6** concludes the report by summarizing the project outcomes, discussing system limitations, and outlining potential future enhancements and research directions for TourMate AI.

Chapter (2) Market and Literature Survey

2.1 Overview of Current Tourism Planning Apps

The way people plan travel is changing. In the past, travel websites were just tools to book flights or hotels. They were effective for transactional booking but offered limited support for the cognitive effort involved in end-to-end trip planning. The user still had to figure out where to go, how long to stay, and how to go from one place to another.

Recently, Artificial Intelligence (AI) has improved travel apps. However, most new AI tools are just chatbots. They give lists of suggestions, but they don't understand real-world limits like time or distance.

Agentic AI systems are the next big step. Unlike simple chatbots, an "agentic" system acts like a smart assistant. It can plan a trip, check if it works, and improve it based on your feedback. TourMate AI is built on this new technology. It is designed to be more than just a search engine; it is a smart partner that helps users build complete, realistic travel plans.

2.2 Related Work: Key Applications in the Market

This section looks at the most popular travel apps used today. We compare them to TourMate AI to show why a new solution is needed.

2.2.1 Vacay (The Chatbot)

Key Features:

- **Chat-Based Interaction:** Users can request itineraries and modifications using natural language.
- **Quick Itinerary Generation:** Produces multi-day travel plans within seconds.
- **Popular Destination Knowledge:** Suggests well-known landmarks, attractions, and cultural sites.
- **External Booking Links:** Provides links to third-party platforms for flights and accommodations.

- **Conversation History Listing:** Displays previous chat prompts, allowing users to revisit past requests.

Limitations:

- **No Planning Explainability:** The system does not explain why destinations or activities are selected, nor does it justify changes made to the itinerary.
- **Stateless Editing Behavior:** Modifications (e.g., replacing Luxor with Alexandria) result in regenerated text rather than incremental edits, requiring the user to mentally reconcile changes.
- **Lack of Feasibility Checking:** Vacay does not validate travel time, transportation logistics, or daily pacing, which may lead to unrealistic itineraries.
- **Static Text Output:** Itineraries are presented as blocks of text with no visual timeline, drag-and-drop editing, or structural manipulation.
- **No Persistent User Memory:** Although chat history is visible, the system does not retain preferences or planning rationale across sessions.
- **Informational-Only AI Assistant:** The AI does not have the ability to directly edit or optimize itinerary structures.
- **Accuracy Disclaimer:** The platform explicitly warns that AI-generated information and links may be outdated or inaccurate, reducing reliability for detailed trip execution.

2.2.2 Wanderlog (The Digital Notebook)

Key Features:

- **Organizational Tools:** Interactive map and clear daily timeline.
- **Collaborative Planning:** Multiple users can edit the same itinerary.
- **Place Saving:** Bookmark and store locations for later reference.
- **Informational AI Assistant:** Provides guidance without direct control over plans.

Limitations:

- **High Manual Effort:** Users manage order, timing, and structure.
- **No Optimization:** Itineraries aren't evaluated for feasibility or efficiency.
- **Limited AI:** Suggestions lack reasoning or validation.
- **Text-Based Search Only:** No image-based discovery or visual inspiration

2.2.3 Roam Around (The Instant Planner)

Key Features:

- **Very Fast:** Generates multi-day itineraries within seconds.
- **Minimal Input:** Requires only basic information such as city and trip duration.
- **Place Listings:** Provides a list of popular attractions with basic details and map links.

Limitations:

- **Rigid Output:** The system generates a single static itinerary that cannot be easily edited or refined.
- **No Feasibility Reasoning:** It does not consider travel time, daily pacing, or real-world constraints.
- **No Personalization or Learning:** The same itinerary structure is produced for different users, with no memory or adaptation to user preferences.

2.2.4 Guide Geek (The Text Message Helper)

Key Features:

- **High Accessibility:** Works directly inside popular messaging applications without requiring a separate platform.
- **Conversational Interaction:** Responds dynamically to user preferences.
- **Idea Generation:** Provides rich narrative descriptions and inspirational travel suggestions.

Limitations:

- **Unstructured Output:** Recommendations are delivered as long text messages, making them difficult to convert into a clear daily schedule.
- **Context Loss:** In extended conversations, the system often forgets earlier constraints or preferences provided by the user.
- **No Feasibility Validation:** Suggestions are not checked for realism, travel time, or overall trip coherence.

2.2.5 Google Travel (The Big Data Apps)

Key Features:

- **Extensive Data Coverage:** Provides access to a vast database of flights, hotels, destinations, and user reviews.
- **Price Comparison:** Uses historical price trends, alerts, and flexible date tools to help users find cheaper options.
- **Decision Support Tools:** Offers price tracking, booking insights, and AI-based deal discovery.

Limitations:

- **Information Overload:** Users must manually analyze large volumes of options and reviews to make decisions.
- **No Integrated Planning:** The platform does not generate end-to-end itineraries; users must independently combine flights, accommodations, and activities.
- **Lack of Personal Context:** Recommendations are primarily data-driven and do not adapt to individual travel styles or preferences.

2.3 Comparative Analysis

Table 2.3.1 Feature Comparison of AI Travel Planning Platforms

Feature	Vacay	Wanderlog	Roam Around	Guide Geek	TourMate AI
AI-Generated Itineraries	✓	✓	✓	✓	✓
Structured Daily Plans	✓	✓	✓	✓	✓
Easy Editing & Refinement	✓	✗	✓	✓	✓
Multi-Modal Input (Text + Images)	✗	✗	✗	✗	✓
Behavioral Learning System	✗	✗	✗	✗	✓
Candidate Scoring & Ranking	✗	✗	✗	✗	✓
Time & Location Feasibility Checks	✗	✗	✗	✗	✓
End-to-End Trip Optimization	✗	✗	✗	✗	✓
Simulated Booking Execution	✗	✗	✗	✗	✓

2.4 Gaps in the Market

Based on this research, there are four big problems that current apps do not solve:

- **All or Nothing:** Apps either make you do all the work (Wanderlog) or give you a plan you can't change (Roam Around). There is no middle ground where the AI helps, but the user decides.
- **Ignoring Images:** Travel is visual. Current apps only understand text. They cannot look at a photo users like and understand the "style" or "vibe" of that photo.
- **Not Ranked or Grounded in Reality:** Current apps often treat every suggested place with equal weight, rather than ranking suggestions based on how well they match the traveler's own interests. They also rarely check whether the resulting plan is actually realistic in terms of timing, distance, and pacing between stops.
- **No Memory:** Most bots treat every session like a new start. They don't remember the user's past trips or preferences.

2.5 How TourMate AI Solves These Problems

TourMate AI is designed to fix these specific gaps, while also introducing additional capabilities that further set it apart from existing solutions.

- **Smart Agents (Solving Gap 1):** TourMate uses "agents" that think like a human planner. They create a plan, check if it works, and then ask the user for approval. This gives the user the best of both worlds: less work, but full control.
- **Visual Understanding (Solving Gap 2):** TourMate uses a Vision-Language Model (VLM). This means users can upload photos (like a picture of a beach or a cozy cafe), and the AI understands the style the user wants.
- **Smart Ranking & Real-World Logic (Solving Gap 3):** TourMate AI ranks candidate places according to how well they match the traveler's profile and interests, and validates the resulting plan against time and location constraints to ensure it is realistic and feasible, not just a random list of places.

- **Learning Profile (Solving Gap 4):** TourMate remembers the user. When a trip is approved, the system updates the traveler's profile based on the choices that were made. Over time, this builds a richer picture of the user's preferences, enabling better suggestions for future trips.
- **Integrated Travel Logistics:** Beyond itinerary planning, TourMate AI supports searching, comparing, and reserving flights, as well as selecting accommodations that match the traveler's preferences, directly within the same conversational flow — closing a gap that most itinerary-only tools leave to external booking platforms.
- **Transparent Decision-Making:** Every recommendation and itinerary adjustment comes with a clear explanation of why it was made, giving users visibility into the reasoning behind the plan rather than an unexplained list of suggestions.

2.6 Conclusion

Travelers today don't just want data; they want help using it. They want a tool that reduces stress but keeps them in control.

TourMate AI fills this need. It acts as an Intelligent Travel Co-Pilot. By combining the organization of manual planners with the creativity of AI—and adding the ability to understand images—TourMate AI proposes a more comprehensive approach compared to existing solutions.

Chapter (3) TourMate AI Analysis

3.1 Functional Requirements

ID	Requirement Description
1. Authentication & Account Management	
#01	The system allows users to register and log in securely using email/password authentication.
#02	Users shall maintain their session through automatic token refresh.
#03	User profile information and travel preferences shall be manageable through direct edits or automatically refined through system interactions based on approved trip data.
2. Intelligent User Profiling & Personalization	
#04	The system shall maintain an evolving profile per trip that automatically refines recommendations based on the user's ongoing interactions, approved plans, and in-session behavior.
#05	Each new trip shall begin with a default profile that improves over time as the user makes choices, adjusts plans, and provides feedback during planning.
#06	Every generated itinerary shall be tailored to the trip's budget, travel style, pace, interests, and accommodation preferences as expressed through the conversation.
#07	The trip profile shall remain internal and invisible to the user, used solely to personalize that specific trip's recommendations and itinerary.
3. Conversational & Multimodal Trip Requests	
#08	Users shall be able to plan trips by describing their requirements in natural language through a real-time chat interface.
#09	The system shall accept both text and image inputs, extracting travel preferences from photos to enhance trip recommendations.

#10	When critical information is missing, the system shall ask clarifying questions until all requirements are clear before proceeding
4. AI-Powered Itinerary Generation & Editing	
#11	The system shall generate complete day-by-day itineraries that are logically sequenced, time-efficient, and matched to the user's profile.
#12	Generated itineraries shall be validated for quality and automatically improved through retry mechanisms when needed.
#13	Users shall be able to modify their itinerary using natural language — whether fine-tuning preferences, adding/removing specific activities, or requesting full regeneration.
5. Place Discovery & Exploration	
#14	Users shall be able to browse and discover places (attractions, restaurants, hotels) with photo previews, ratings, and key information.
#15	Places shall be filterable by category, city, and accommodation type so users can find exactly what they're looking for.
#16	Users shall be able to search for places using natural language descriptions in addition to structured filters.
6. Flight & Hotel Booking	
#17	Users shall be able to book flights and hotels through a secure payment flow, with explicit confirmation required before any booking is finalized.
#18	A simulated payment mode shall be available for development and testing without real financial transactions.
7. Reviews & Personal Bookmarks	
#19	Users shall be able to rate and review places they have visited, with one review allowed per user per place, which they may later edit or delete.

#20	Users shall be able to like reviews written by others and view their own review history.
#21	Users shall be able to bookmark places for future reference, with the ability to add personal notes to saved places.
8. Image-Based Travel Inspiration	
#22	Users shall be able to upload travel photos that the system analyzes to extract travel style cues and interests.
#23	Extracted preferences from images shall be incorporated into trip planning when confidence is medium or higher.
9. Trip Lifecycle Management	
#24	Users shall be able to create, view, update, and cancel trips throughout their lifecycle.
#25	Trip status shall progress through the following states: planning → itinerary draft → awaiting booking → booking pending → payment processing → payment failed (a recoverable state allowing retry) → booking confirmed → active → completed, with cancelled available as a terminal state reachable when a user cancels the trip.
#26	Users shall explicitly approve itineraries before advancing to the booking phase.
#27	Failed payments shall be recoverable, allowing users to retry without losing progress.
10. Conversation Flow & Context Management	
#28	The system shall maintain conversation context across chat sessions, remembering previous interactions and user choices.
#29	Each message shall be interpreted to determine the user's intent and routed to the appropriate feature.
#30	In-progress conversations shall be preserved if disconnected, allowing users to resume where they left off.

11. Transparency & Continuous Improvement	
#31	The system shall track AI actions and performance metrics to support quality assurance, debugging, and operational monitoring.
#32	User feedback on completed trips and itineraries shall be collected to improve future recommendations.
#33	All AI operations shall be observable for monitoring performance, latency, and resource usage.
12. System Health & Monitoring	
#34	The system shall expose a health check endpoint for monitoring service availability.
#35	Performance metrics (response times, error rates, usage patterns) shall be tracked for operational visibility.

3.2 Non-Functional Requirements

Non-functional requirements define quality standards and operational constraints that ensure the TourMate AI system is secure, efficient, and reliable.

Performance:

- **Response Latency:** The system shall return conversational (non-generation) responses within 2–3 seconds and shall complete a full itinerary generation request within 20 seconds under normal operating conditions.
- **Concurrency:** The system shall support at least 50 concurrent active user sessions without measurable degradation in response time.

Security:

- **Data Protection:** The system shall never store user credentials directly; all stored personal and preference data shall be encrypted in transit and at rest.
- **Access Control:** Authenticated sessions shall remain valid for up to 1 hour before requiring silent renewal, and idle conversation sessions shall expire after 1 hour of inactivity.

Usability:

- **Intuitive Design:** The interface shall visually acknowledge every user action within 1 second, independent of backend processing time.
- **Error Handling:** If an AI-driven request cannot be completed after 3 automatic retry attempts, the system shall return a clear, user-facing fallback message within 2 seconds rather than leaving the request pending indefinitely.

Reliability & Availability:

- **Data Persistence:** Trip, itinerary, and conversation history data shall be retained indefinitely; only transient session state may expire, and never sooner than 1 hour after the last interaction.
- **System Stability:** Transient failures in any system dependency shall trigger automatic retry, up to 3 attempts, before the system degrades gracefully with an explanatory message rather than failing the entire request.

Scalability & Maintainability:

- **Architectural Flexibility:** The system shall support at least 4 destination cities at launch, with the ability to onboard additional cities without modifying core system logic.
- **Decoupling:** The system shall allow the AI model or provider to be used for any given task to be replaced or upgraded without requiring changes to other system components.

3.3 User Personas

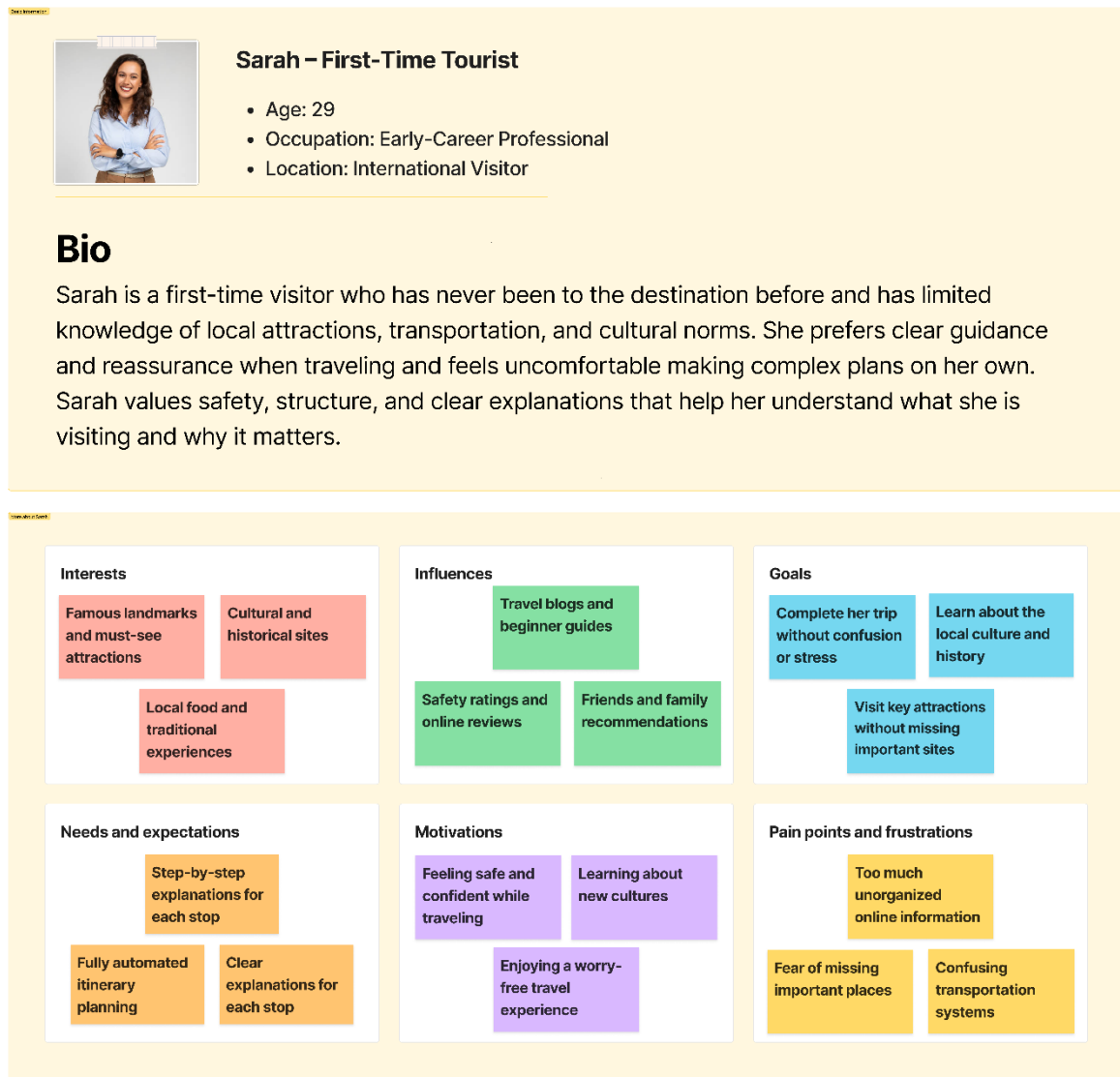


Figure 3.3.1 First-Time Tourist Persona

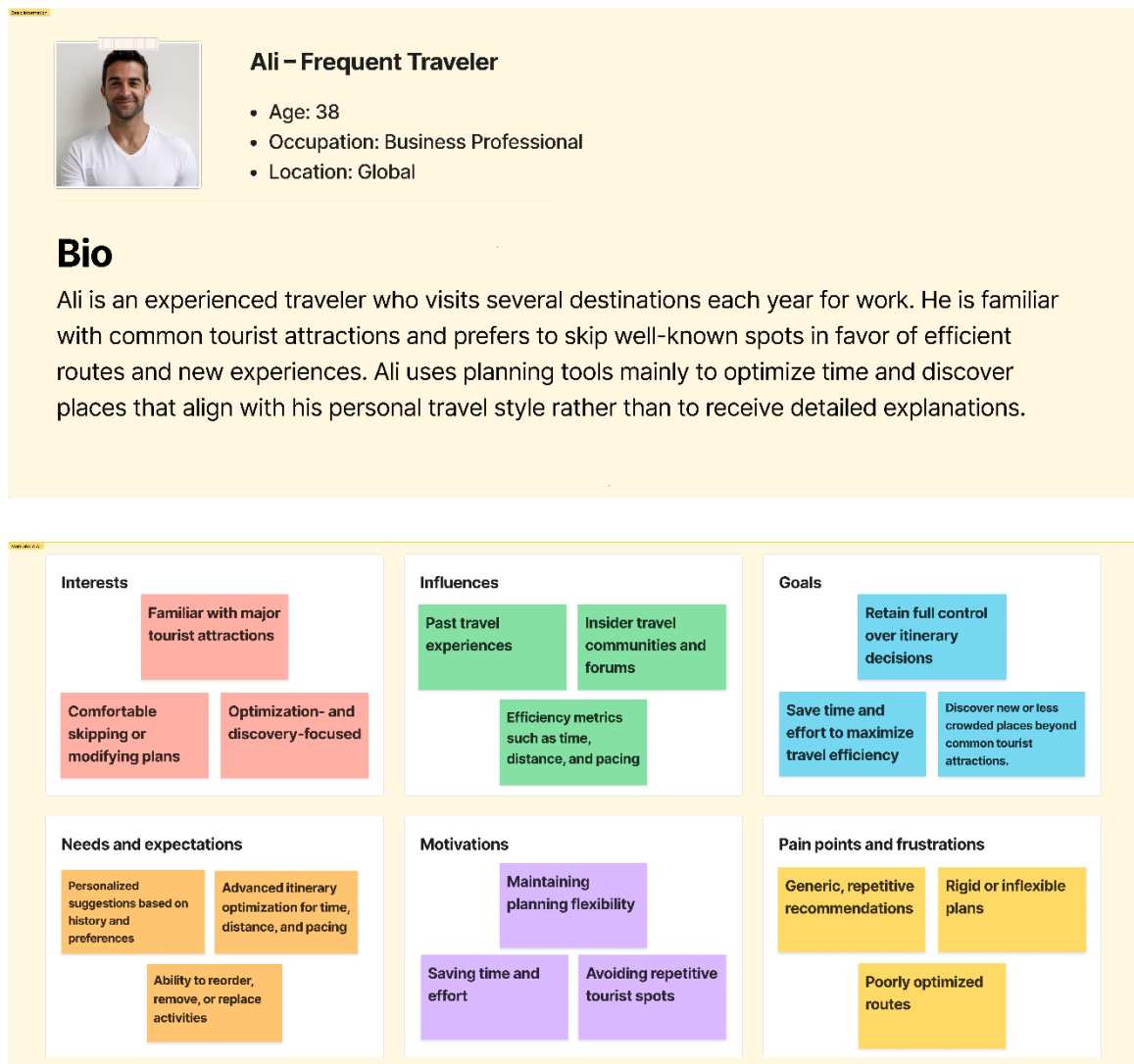


Figure 3.3.2 Frequent Traveler Persona

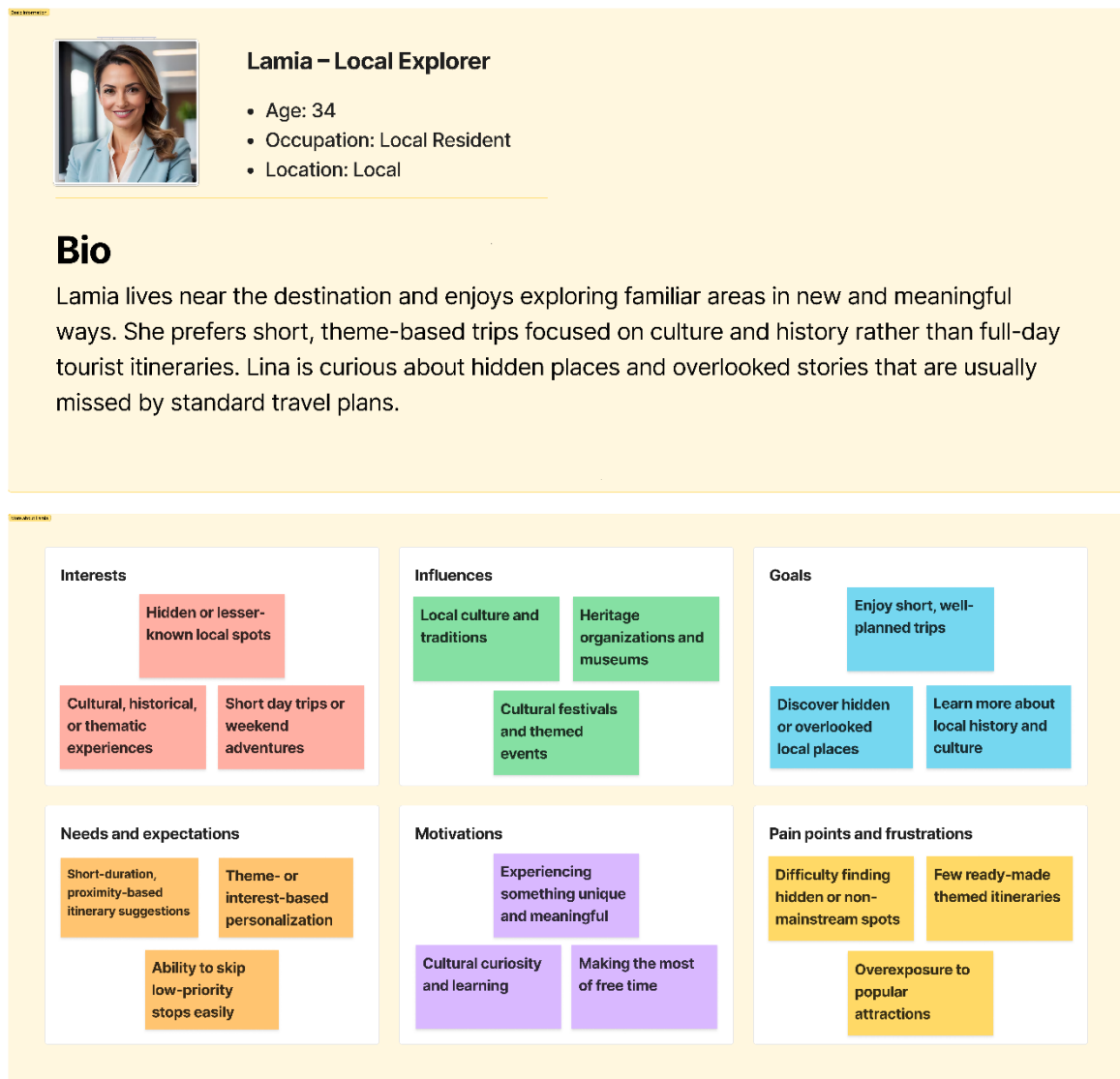


Figure 3.3.3 Local Explorer Persona

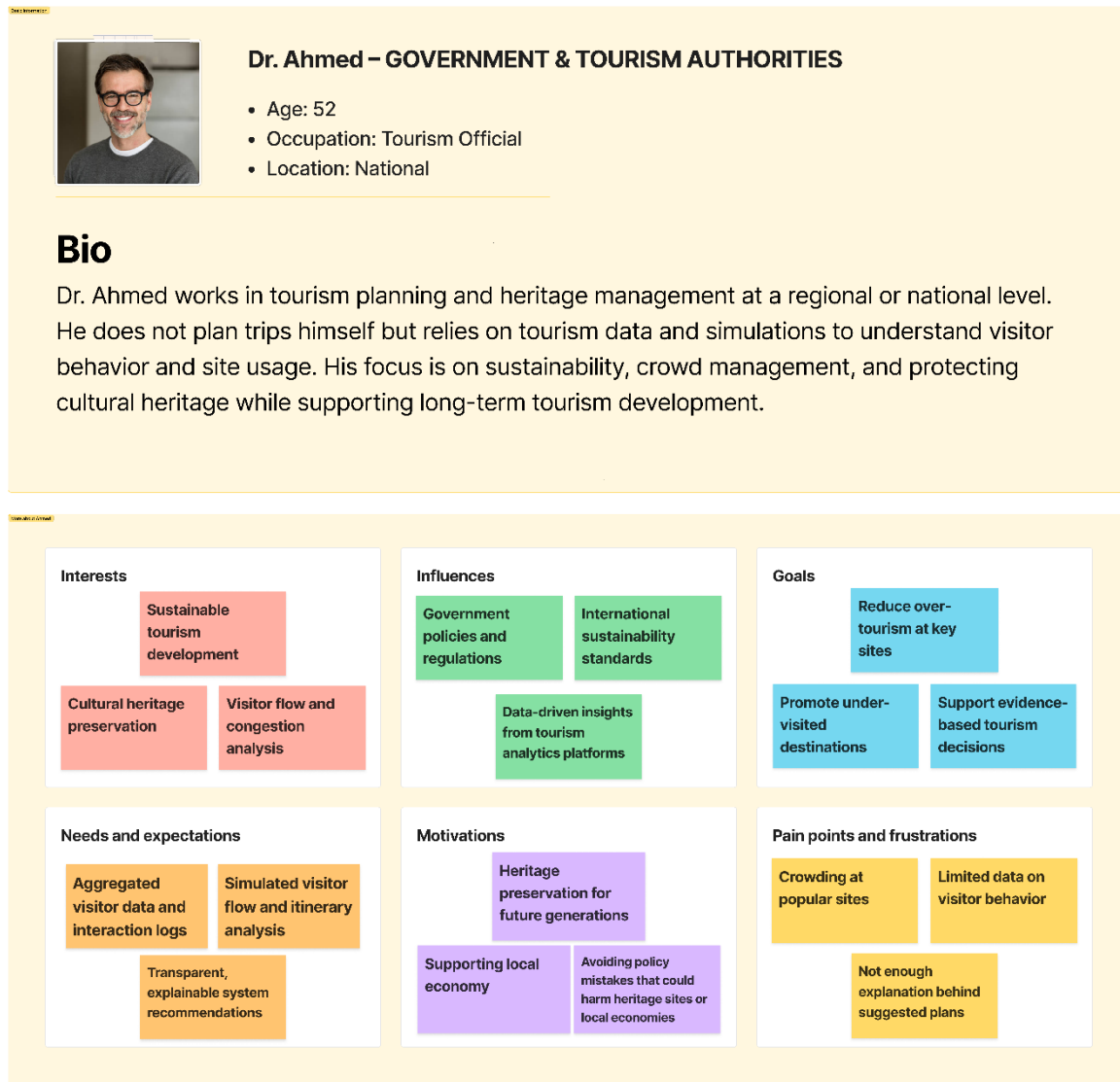


Figure 3.3.4 Government & Tourism Authorities Persona

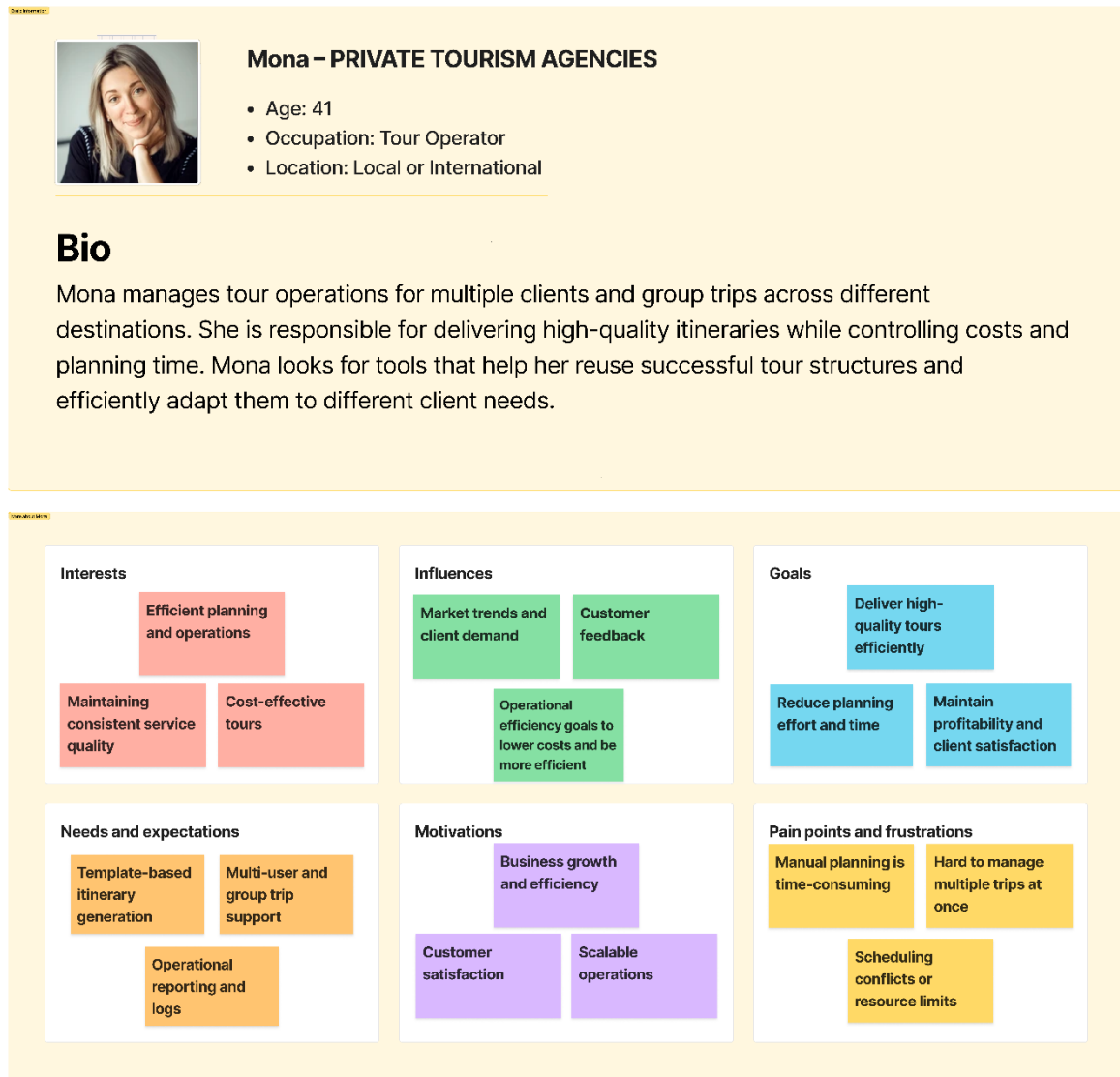


Figure 3.3.5 Private Tourism Agencies Persona

3.4 Use-Case Diagram

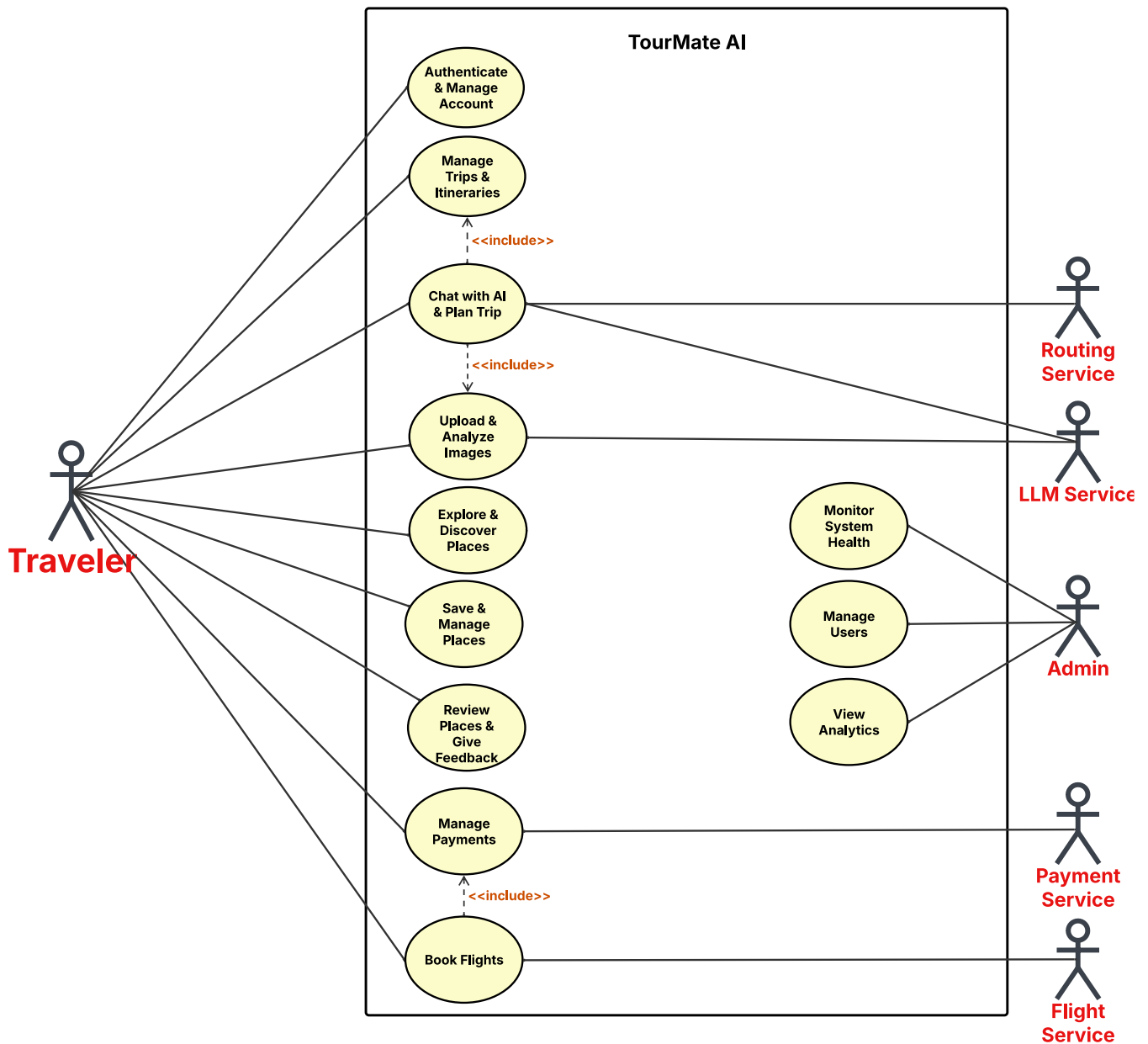


Figure 3.4.1 Use Case Diagram

3.5 Use-Case Narratives

Table 3.5.1 Authenticate & Manage Account Scenario

Use Case ID	UC#1	
Use Case Name	Authenticate & Manage Account	
Actors	Traveler	
Pre-Condition(s)	The traveler has not yet signed up or logged in.	
Post-Condition(s)	The traveler is authenticated and can access all protected features of the platform.	
Main Normal Flow (Sign Up)	Traveler	System
	1- The traveler creates an account by providing their email address and a password.	
		2- The system verifies their identity and creates a secure account.
	3- The traveler is logged in and can start using the platform.	
Alternative Flow A (Login)	Traveler	System
	1- A returning traveler logs in using their email and password.	
		2- The system verifies the credentials and grants access.

	3- The traveler's session is maintained so they don't need to log in repeatedly.	
Alternative Flow B (Session Continuation)	Traveler	System
	1- If the traveler's session expires, the system automatically refreshes it without requiring re-login.	
		2- The traveler continues using the platform seamlessly.profile
Alternative Flow C (Login Failure)	Traveler	System
	1- the traveler enters incorrect credentials, or a network issue occurs	
		2- the system notifies them and allows them to retry.
Exceptions	1- Network issues prevent authentication — the traveler is asked to try again later. 2- The traveler tries to sign up with an email that is already registered — the system prompts them to log in instead.	

Table 3.5.2 Manage Trips & Itineraries Scenario

Use Case ID	UC#2	
Use Case Name	Manage Trips & Itineraries	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in.	
Post-Condition(s)	A trip is created and moves through its natural lifecycle (planning, itinerary ready, booking, in progress, completed, or cancelled).	
Main Flow (Create & View Trip)	Traveler	System
	1- The traveler starts a conversation with the AI and describes the trip they want — for example, "Plan me a weekend trip to Cairo."	
		2- The AI creates a new trip with the chosen destination and dates, and the trip begins in the planning phase.
	3- The traveler can view their active trips at any time through the trips list, with full details about each trip's itinerary, bookings, and status.	
Alternative Flow A (Approve Itinerary)	Traveler	System
		1- The AI generates a day-by-day itinerary and presents it directly in the chat conversation.
	2- The traveler reviews the itinerary in the chat and gives approval by typing their confirmation.	

		3- The itinerary is marked as approved and the trip moves toward the booking phase, where flight and hotel selection becomes available through the chat.
Alternative Flow B (Cancel Trip)	Traveler	System
	1- The traveler decides to cancel a trip at any stage by requesting it through the chat.	
		2- The system marks the trip as cancelled and releases any associated bookings.
		3- The trip is removed from the active trips list.
Alternative Flow C (View Trip Details)	Traveler	System
	1- The traveler selects a specific trip from their trips list to see its full details.	
		2- The system shows the complete itinerary, booked flights, accommodation, and payment status.
Notes	- Trips are created through the chat conversation when the traveler first describes their trip to the AI. - Itinerary approval happens within the chat — the traveler simply confirms their approval in conversation. - A trip progresses through stages: planning → itinerary ready → awaiting booking → booking pending → payment processing → booking confirmed → in progress → completed. A trip may also be cancelled and removed from the system at any stage prior to completion. - If a payment fails, the trip enters a dedicated recoverable state so the traveler can retry without losing progress. - Each trip contains one itinerary with multiple days and stops.	

Table 3.5.3 Chat with AI & Plan Trip Scenario

Use Case ID	UC#3	
Use Case Name	Chat with AI & Plan Trip	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in and connected to the chat session.	
Post-Condition(s)	The AI generates a personalized day-by-day itinerary based on the traveler's preferences.	
Main Flow (Plan a New Trip)	Traveler	System
	1- The traveler opens the chat and describes the desired trip in natural language—for example, "Plan me a 2-day trip to Cairo."	
		2- The AI interprets the request and checks whether all essential trip details (such as destination, duration, travel style, and budget) are available.
		3- If any information is missing, the AI asks follow-up questions until the request is complete.
		4- The AI identifies suitable attractions, restaurants, hotels, and activities based on the traveler's preferences and generates a personalized day-by-day itinerary.
		5- The completed itinerary is presented to the traveler through the chat interface.

Alternative Flow A (Refine Preferences)	Traveler	System
	1- The traveler requests preference changes—for example, "Make this trip more cultural."	
		2- The AI updates the itinerary to better reflect the revised preferences and presents the updated plan.
Alternative Flow B (Edit a Specific Activity)	Traveler	System
	1- The traveler requests a specific change—for example, "Add a museum to Day 1" or "Remove the Eiffel Tower."	
		2- The AI modifies the requested activity while preserving the rest of the itinerary whenever possible.
		3- The updated itinerary is presented to the traveler.
Alternative Flow C (Regenerate the Entire Plan)	Traveler	System
	1- The traveler decides they want a completely different itinerary—for example, "Actually, I hate museums. Regenerate the trip without them."	
		2- The AI generates a new itinerary based on the updated preferences and presents it to the traveler.

Alternative Flow D (General Conversation)	Traveler	System
	1- The traveler asks a general question, such as "What's the weather like in Paris?"	
		2- The AI responds conversationally without modifying the current itinerary.
Alternative Flow E (Use a Photo for Inspiration)	Traveler	System
	1- The traveler uploads a travel photo to express their preferred style or interests.	
		2- The AI analyzes the image and uses the identified preferences to personalize the itinerary recommendations.
Exceptions	- If the AI service is temporarily unavailable, the system falls back to an alternative provider. - If the chat session is interrupted, the conversation state is preserved so the traveler can resume where they left off.	
Notes	- The AI supports several editing modes depending on the scope of the requested change, ranging from quick preference or accommodation adjustments, to specific activity edits, up to full itinerary regeneration. - All AI actions are tracked for quality assurance and cost monitoring. - The itinerary is validated both programmatically and by AI before being presented.	

Table 3.5.4 Explore & Discover Places Scenario

Use Case ID	UC#4	
Use Case Name	Explore & Discover Places	
Actors	Traveler	
Pre-Condition(s)	The database contains attractions, restaurants, and hotels. The traveler may or may not be logged in.	
Post-Condition(s)	The traveler finds places of interest and can view their full details.	
Main Flow (Browse Explore)	Traveler	System
	1- The traveler opens the Explore tab and sees suggested locations with category tabs (attractions, restaurants, hotels).	
		2- The system displays a paginated list of places with photos, ratings, and basic information.
	3- The traveler scrolls through results and browses at their own pace.	
Alternative Flow A (Filter by Category)	Traveler	System
	1- The traveler selects a category tab — for example, "Restaurants."	
		2- The system shows only places belonging to that category.
		3- For hotels, results can further be distinguished by accommodation type (hotel, hostel, resort) to support more targeted browsing.

Alternative Flow B (Search by City)	Traveler	System
	1- The traveler types a city name — for example, "Cairo."	
		2- The system suggests matching locations, and the traveler selects one.
		3- The system displays places in that city.
Alternative Flow C (View Place Details)	Traveler	System
	1- The traveler taps on a place card to learn more.	
		2- The system shows full details: description, photos, address, location on map, pricing, and ratings.
Alternative Flow D (Search in Natural Language)	Traveler	System
	1- The traveler searches using descriptive text — for example, "romantic dinner with sea view."	
		2- The system finds places that best match the description and ranks them by relevance.
Notes	- Browsing places is publicly available without needing an account. - Categories available: attractions, restaurants, and hotels. - The AI engine uses the same place search and accommodation type filtering internally when selecting lodging options during trip planning.	

Table 3.5.5 Book Flights Scenario

Use Case ID	UC#5	
Use Case Name	Book Flights	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in, has a planned trip, and the itinerary has been approved.	
Post-Condition(s)	A flight is booked and confirmed after successful payment.	
Main Flow (Search, Select, Book, Pay)	Traveler	System
	1- After the itinerary is approved, the AI presents available flight options within the chat, with the destination and travel dates pre-filled from the trip.	
	2- The traveler provides their departure city.	
		3- The system searches for available flights based on the provided travel details.
		4- The available flight options, including airlines, schedules, and prices, are displayed in the chat.
	5- The traveler selects a preferred flight.	
		6- The system presents a secure checkout for payment.
	7- The traveler completes the payment using their preferred payment method.	

		8- The system confirms the booking and displays the reservation details in the chat.
Exceptions	- If no flights are available for the chosen route, the system informs the traveler in the chat and suggests alternatives. - If payment succeeds but the airline booking fails, the traveler is guided through the refund process within the conversation.	
Notes	- Flight booking is driven through the chat conversation, with the app presenting matching screens for reviewing flight options, confirming a selection, and completing payment. - City names are resolved to airport codes automatically behind the scenes for a smooth experience. - The booking flow uses a two-step process (initiate payment → confirm) to handle mobile payments reliably.	

Table 3.5.6 Manage Payments Scenario

Use Case ID	UC#6	
Use Case Name	Manage Payments	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in and has at least one unpaid booking associated with their trip.	
Post-Condition(s)	The payment is successfully processed, and the corresponding booking(s) are confirmed, or the payment fails and the traveler can retry.	
Main Flow (Pay for a Single Booking)	Traveler	System
		1- After a booking is created, the system presents a secure checkout option within the chat conversation.
	2- The traveler selects a payment method and completes the payment.	
		3- The system processes and verifies the payment.
		4- The booking is confirmed, and the trip status is updated accordingly.
Alternative Flow A (Pay All Pending Bookings at Once)	Traveler	System
	1- The traveler chooses to pay for all pending bookings together.	
		2- The system presents a combined checkout covering all unpaid bookings.
		3- Once the payment is successful, all bookings are confirmed and the trip status is updated.

Alternative Flow B (Pay for Flights and Hotels in One Go)	Traveler	System
	1- The traveler has both flight and hotel bookings and chooses to pay for them together.	
		2- The system processes the payment for all selected bookings.
		3- Each booking is confirmed individually, and the traveler is notified once all confirmations are completed.
Alternative Flow C (Payment Failure Recovery)	Traveler	System
	1- The payment attempt fails.	
		2- The system notifies the traveler that the payment was unsuccessful while preserving the booking.
	3- The traveler retries the payment when ready.	
		4- The system processes the new payment attempt.
Exceptions	- If the payment service is unavailable, the system falls back to a fully simulated payment — no secure connection needed. - If a payment succeeds but the booking confirmation fails (e.g., Amadeus error), the traveler is guided through the refund process.proper retention and archiving strategies - Logs support system monitoring, debugging, compliance, and accountability, particularly for AI-driven decisions	
Notes	- Payment is managed as a separate step after booking — the traveler books first, then pays. - All payments run in a sandbox (test mode) environment, and flight reservations are processed through a corresponding test environment — no real financial transactions or real flight bookings take place.	

	- If Stripe is not configured, the system falls back to a fully simulated payment with no external calls. - Payments can be handled individually per booking or batched together for convenience. - The system processes payments both synchronously (client confirms after payment sheet) and asynchronously (via webhook). - Hotel bookings are simulated; flight bookings reflect real airline data through a test environment, without finalizing an actual reservation.
--	--

Table 3.5.7 Review Places & Give Feedback Scenario

Use Case ID	UC#7	
Use Case Name	Review Places & Give Feedback	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in and has visited a place they want to review.	
Post-Condition(s)	A review with a rating and comment is published for other travelers to see.	
Main Flow (Write a Review)	Traveler	System
	1- The traveler navigates to a place's detail page and views existing reviews.	
	2- The traveler taps "Write a Review" and submits a rating along with a comment.	
		3- The system publishes the review, making it visible to other users.
Alternative Flow A (Update a Review)	Traveler	System
	1- The traveler returns to their existing review and updates the rating or comment.	
		2- The system saves the changes.
Alternative Flow B (Delete a Review)	Traveler	System
	1- The traveler removes their review entirely.	
		2- The system deletes the review and its associated likes.

Alternative Flow C (View My Reviews)	Traveler	System
	1- The traveler opens their personal review history.	
		2- The system shows all reviews they have written across different places.
Alternative Flow D (Like a Review)	Traveler	System
	1- The traveler finds a review helpful and taps the like button.	
		2- The system records the like. Tapping again removes it (toggle).
Exceptions	- If the traveler tries to review the same place twice, the system notifies them that they already have a review. - Travelers can only edit or delete their own reviews.	
Notes	- Only one review per user per place is allowed. - Reviews are shown with the most recent first. - Average ratings are calculated dynamically based on all reviews.together for convenience.	

Table 3.5.8 Save & Manage Places Scenario

Use Case ID	UC#8	
Use Case Name	Save & Manage Places	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in and has found a place they are interested in.	
Post-Condition(s)	The place is saved to the traveler's personal list for future reference.	
Main Flow (Save a Place)	Traveler	System
	1- The traveler is viewing a place — from search results, an itinerary, or the explore page.	
	2- The traveler taps the bookmark or save button.	
		3- The system adds the place to the traveler's saved list.
Alternative Flow A (View Saved Places)	Traveler	System
	1- The traveler opens their saved places list.	
		2- The system shows all bookmarked places with their full details, sorted by most recently saved.
Alternative Flow B (Unsave a Place)	Traveler	System
	1- The traveler taps the unsave button on a previously saved place.	
		2- The system removes it from their saved list.

	Traveler	System
	1- When saving a place, the traveler adds a personal note — for example, "Must visit this!"	
		2- The note is stored alongside the saved place for the traveler's reference.
Notes	- Saved places are personal bookmarks, separate from the itinerary. - Duplicate saves are prevented — a place can only be saved once. - Places are shown most recently saved first.	

Table 3.5.9 Upload & Analyze Images Scenario

Use Case ID	UC#9	
Use Case Name	Upload & Analyze Images	
Actors	Traveler	
Pre-Condition(s)	The traveler is logged in and has a trip to associate images with.	
Post-Condition(s)	Travel photos are uploaded, their content is analyzed, and extracted preferences are used to enhance trip recommendations.	
Main Flow (Upload Photo for Travel Inspiration)	Traveler	System
	1- The traveler uploads a travel-related photo, either during the chat or from their gallery.	
		2- The system stores the image and analyzes it to identify the traveler's interests, travel style, and relevant visual preferences.
		3- The system incorporates the identified preferences into the trip planning process whenever appropriate.
		4- Future itinerary recommendations are personalized based on the extracted preferences.
Exceptions	- If image analysis fails, the system informs the traveler and continues without applying image-based preferences. - Unsupported image formats are rejected with an appropriate message.	
Notes	- Uploaded images help personalize future trip recommendations.	

	- Preferences are only applied when the system's confidence in the extracted style or interests is medium or higher; low-confidence extractions are not used. - Image-based preferences complement, but do not replace, the traveler's explicit preferences.
--	--

Table 3.5.10 Monitor System Health Scenario

Use Case ID	UC#10	
Use Case Name	Monitor System Health	
Actors	System Administrator	
Pre-Condition(s)	The system is running and connected to monitoring tools.	
Post-Condition(s)	The administrator has visibility into the system's health, AI performance, and operational metrics.	
Main Flow (Health Check)	Administrator	System
	1- The administrator or an automated monitoring service checks whether the system is running and responding.	
		2- The system reports its current health status and confirms whether its core services are operational.
Alternative Flow A (Review AI Performance Traces)	Administrator	System
	1- The administrator reviews the execution trace of an AI request.	
		2- The system displays the major processing stages, execution time, and relevant request and response information.
	3- The administrator uses the trace to understand AI behavior or investigate issues.	
Alternative Flow B (Monitor	Administrator	System
	1- The administrator views token usage and cost reports.	

Token Usage & Costs)		2- The system displays token consumption by trip or AI request, helping identify unusual usage patterns and monitor operational costs.
Alternative Flow C (View Performance Metrics)	Administrator	System
	1- The administrator reviews system performance metrics.	
		2- The system displays response times, error rates, request volumes, and historical trends to support capacity planning and performance monitoring.
Alternative Flow D (Review System Logs)	Administrator	System
	1- The administrator searches or filters system logs.	
		2- The system displays errors, warnings, and significant events to help diagnose issues and review system activity.
Notes	- The health check is a lightweight status indicator and does not verify every external dependency. - AI actions are fully traceable for debugging and auditing purposes. - Token usage is tracked per trip to identify which parts of the AI pipeline are most expensive. - All monitoring infrastructure is internal; there is no public admin dashboard.	

Chapter (4) TourMate AI Design and Implementation

4.1 System Architecture

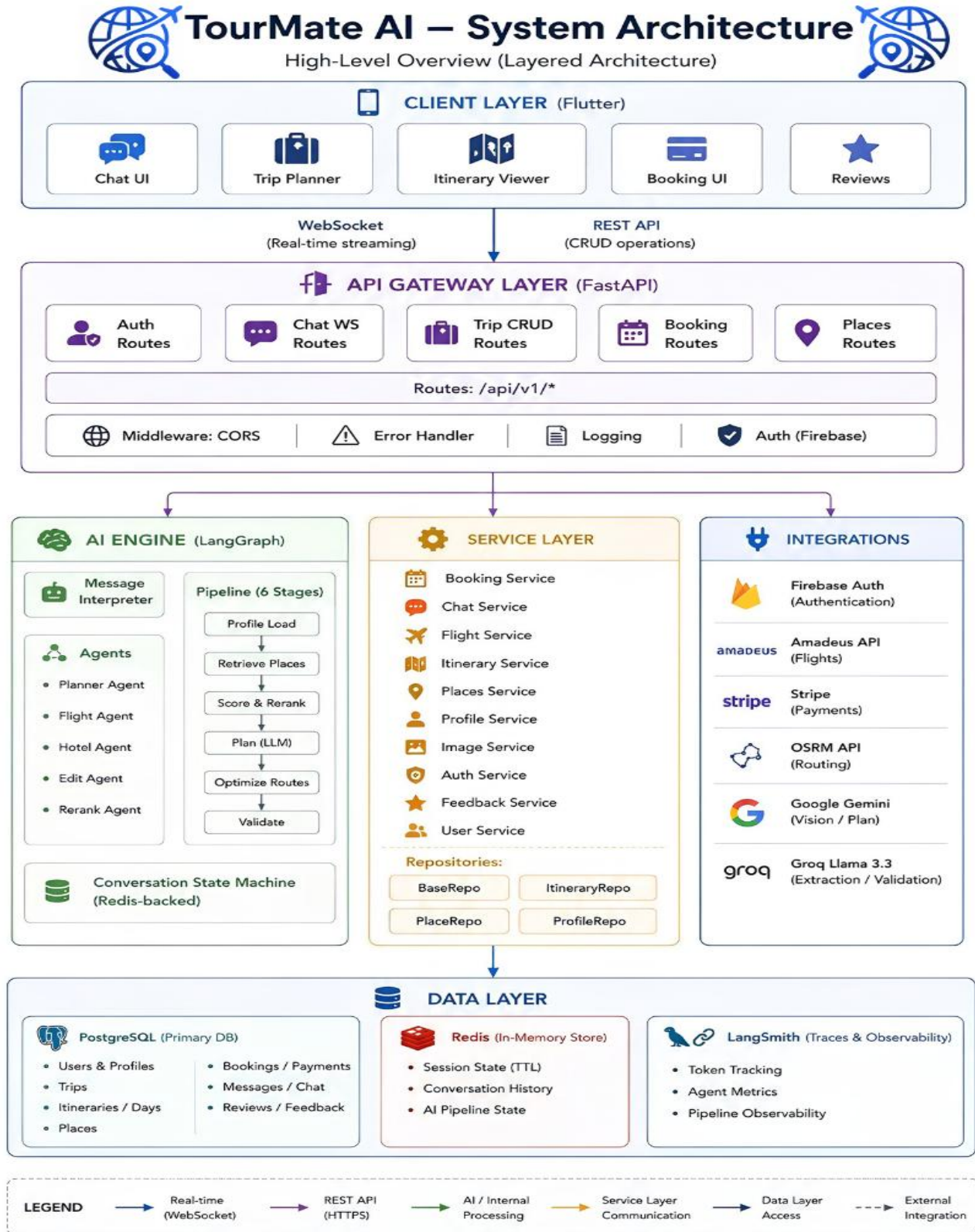


Figure 4.1.1 System Architecture

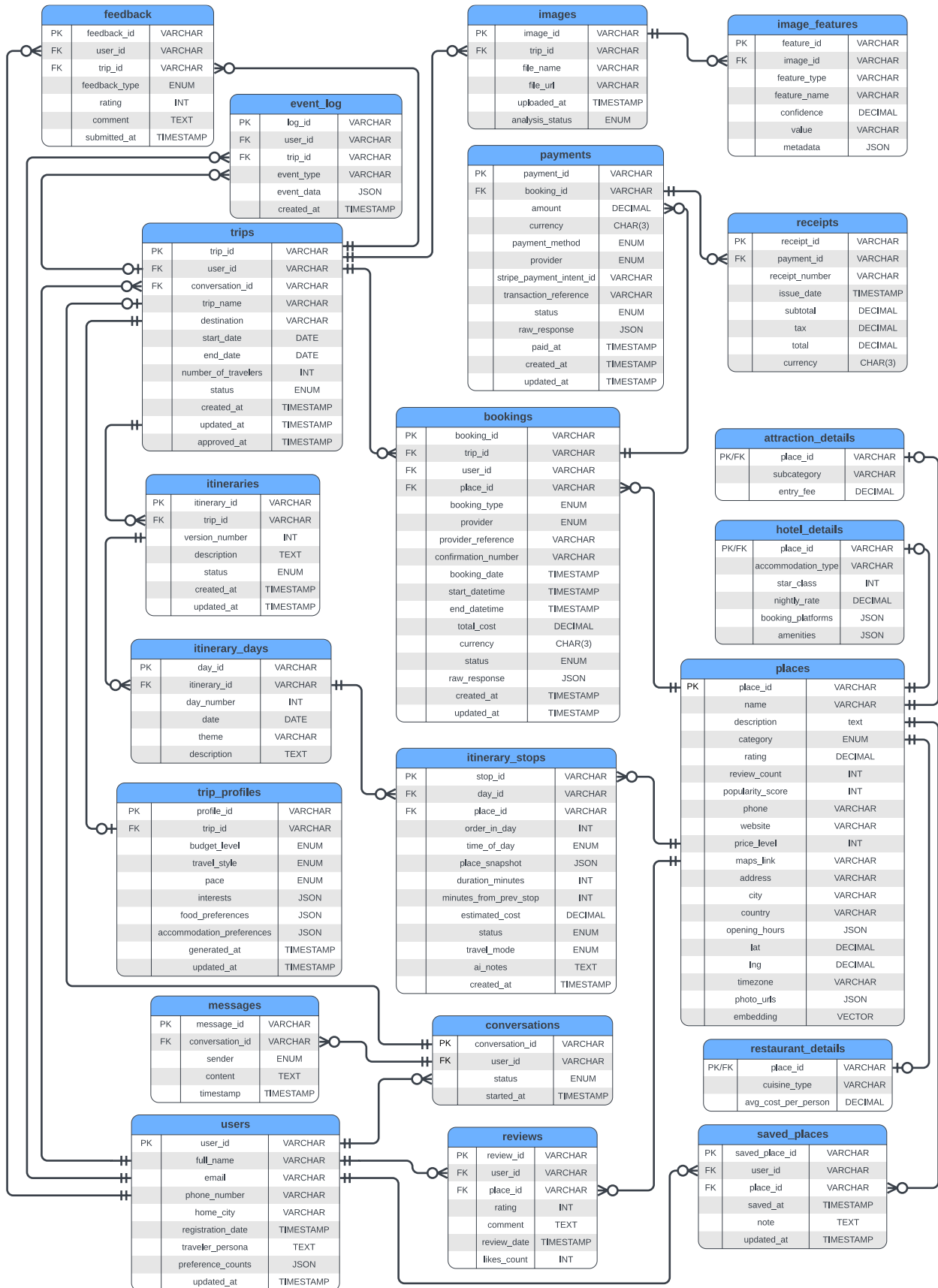
4.2 Entity Relationship Diagram ([view](#))

Figure 4.2.1 Entity Relationship Diagram

4.3 Sequence Diagrams

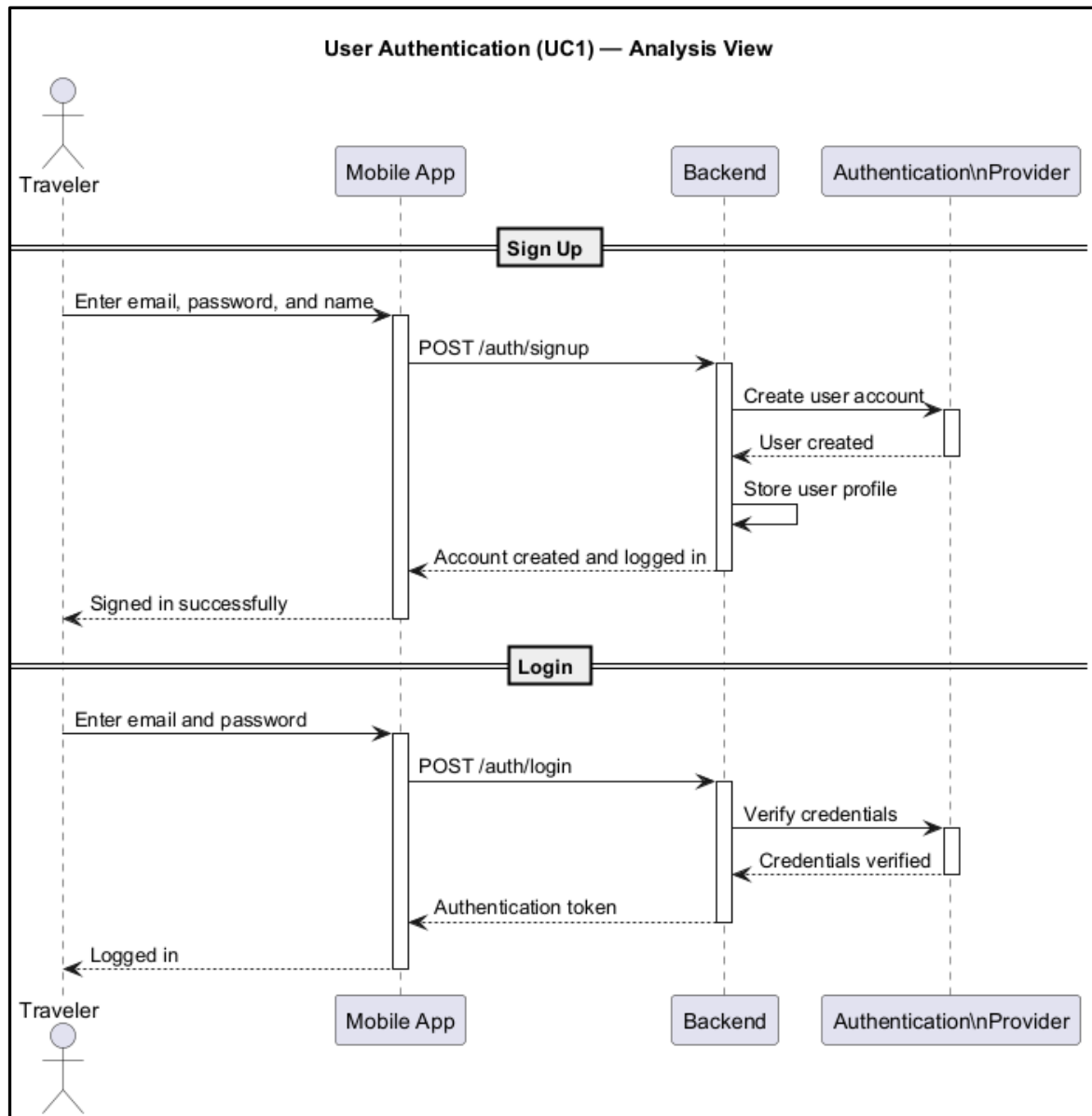


Figure 4.3.1 User Authentication Sequence Diagram

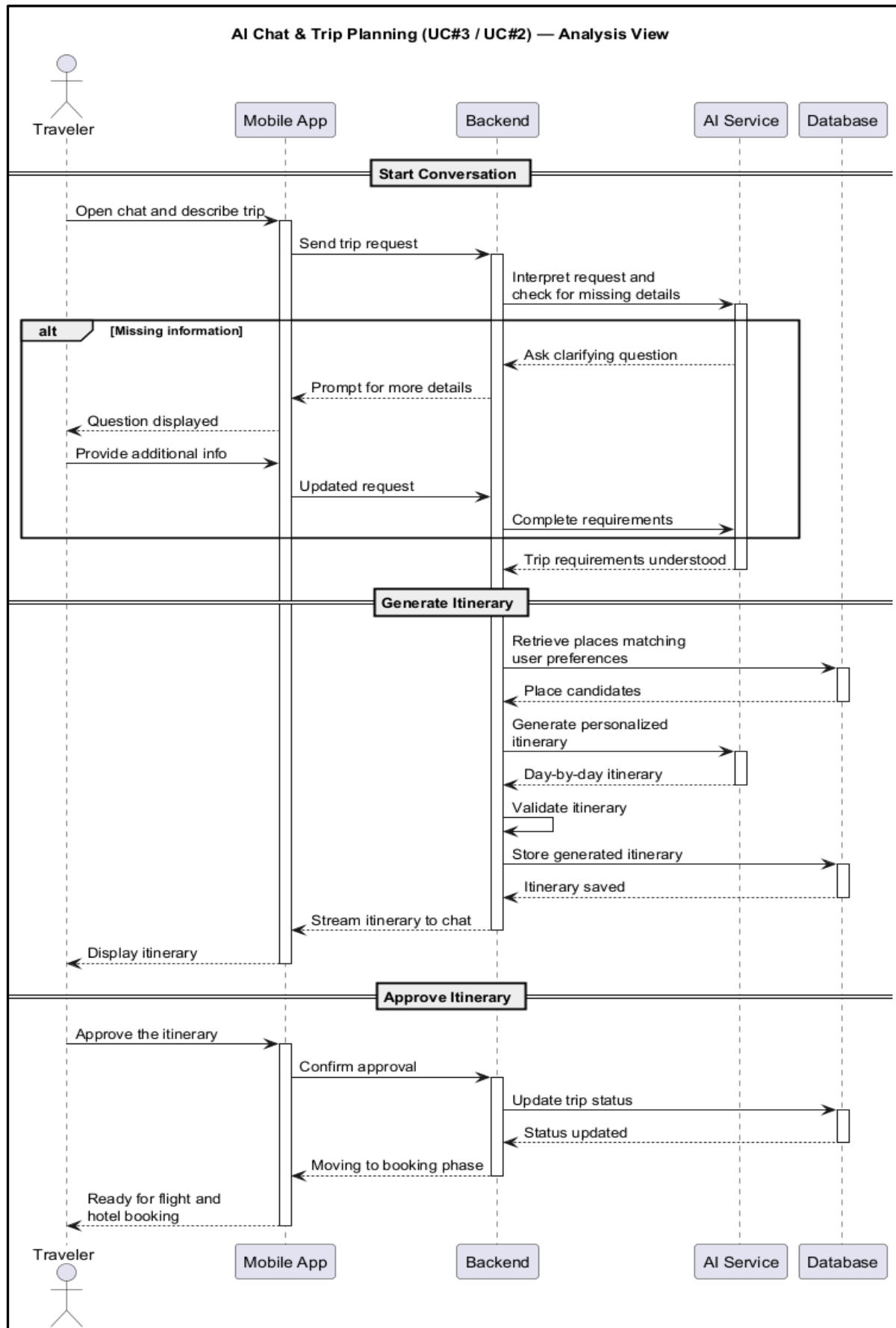


Figure 4.3.2 AI Chat & Trip Planning Sequence Diagram

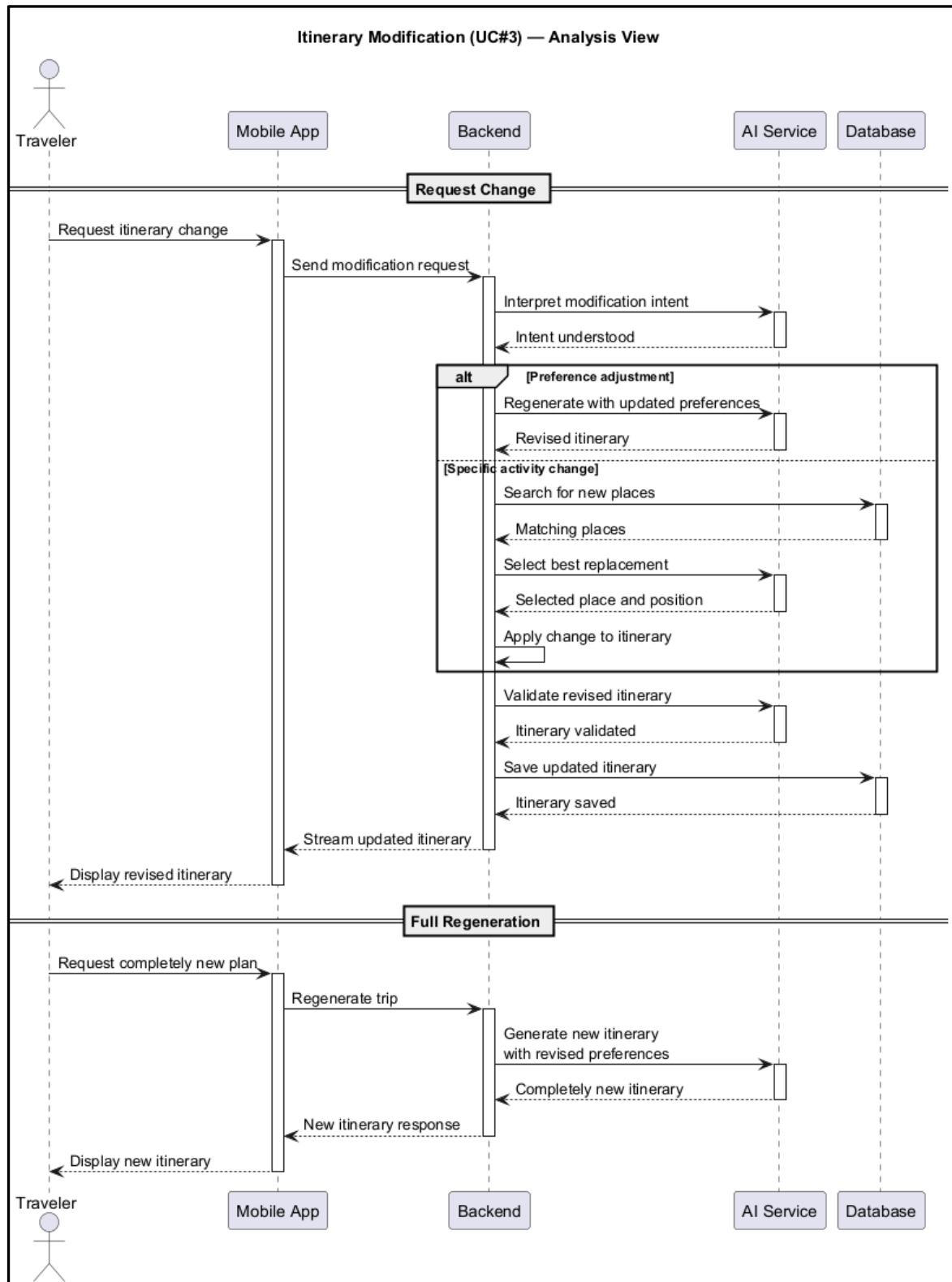


Figure 4.3.3 AI Itinerary Modification Sequence Diagram

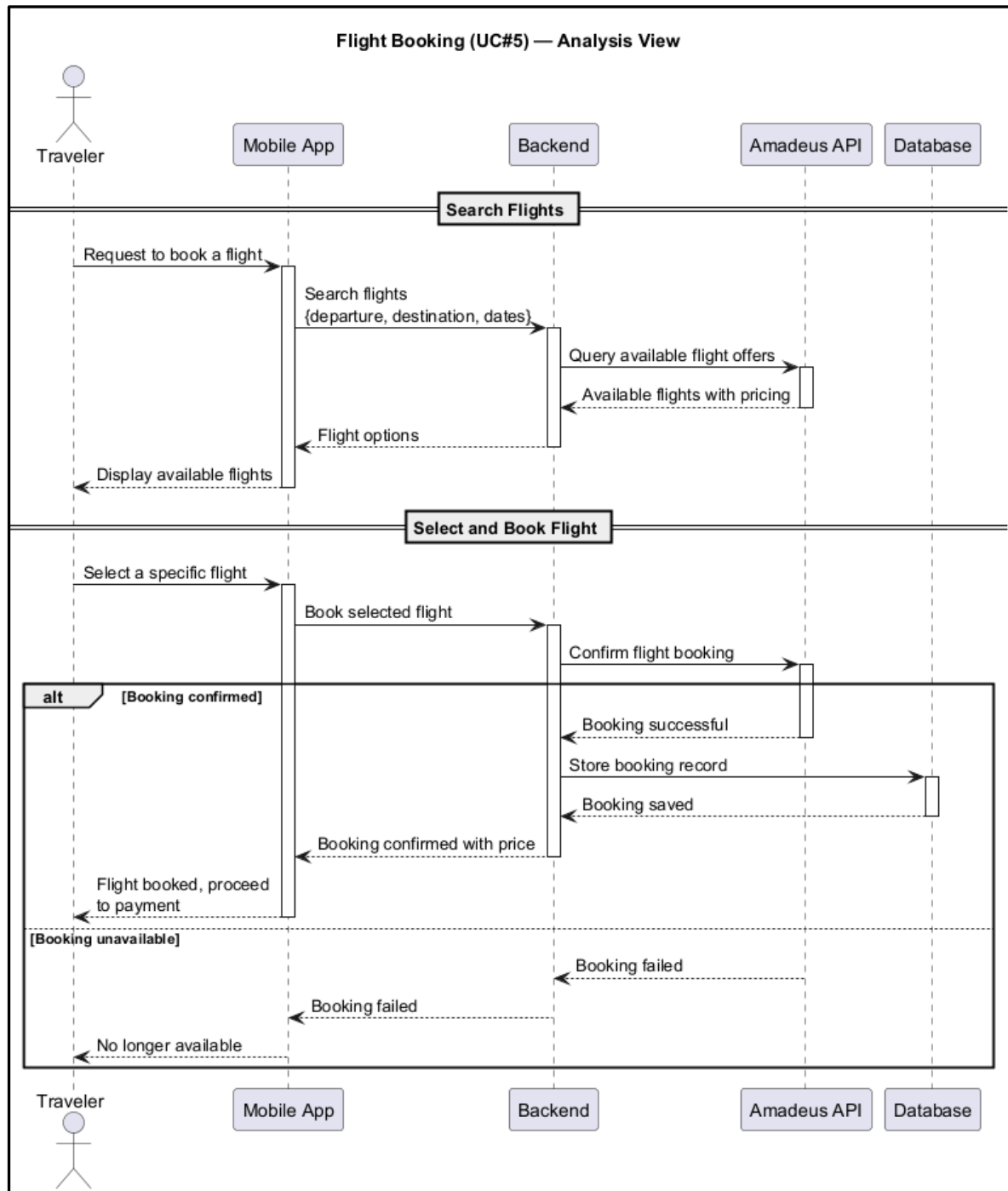


Figure 4.3.4 Flight Booking Sequence diagram

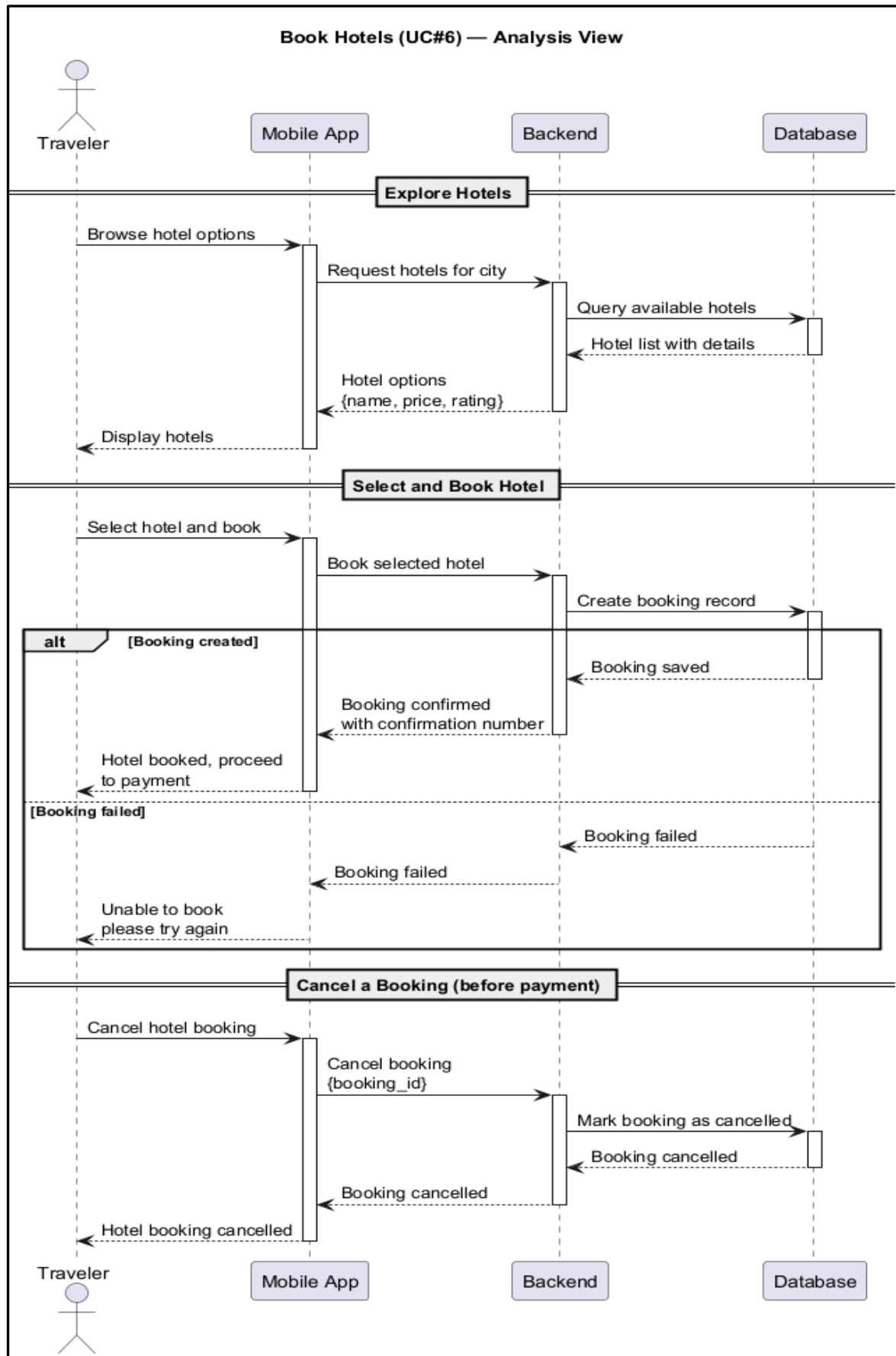


Figure 4.3.5 Hotel Booking Sequence diagram

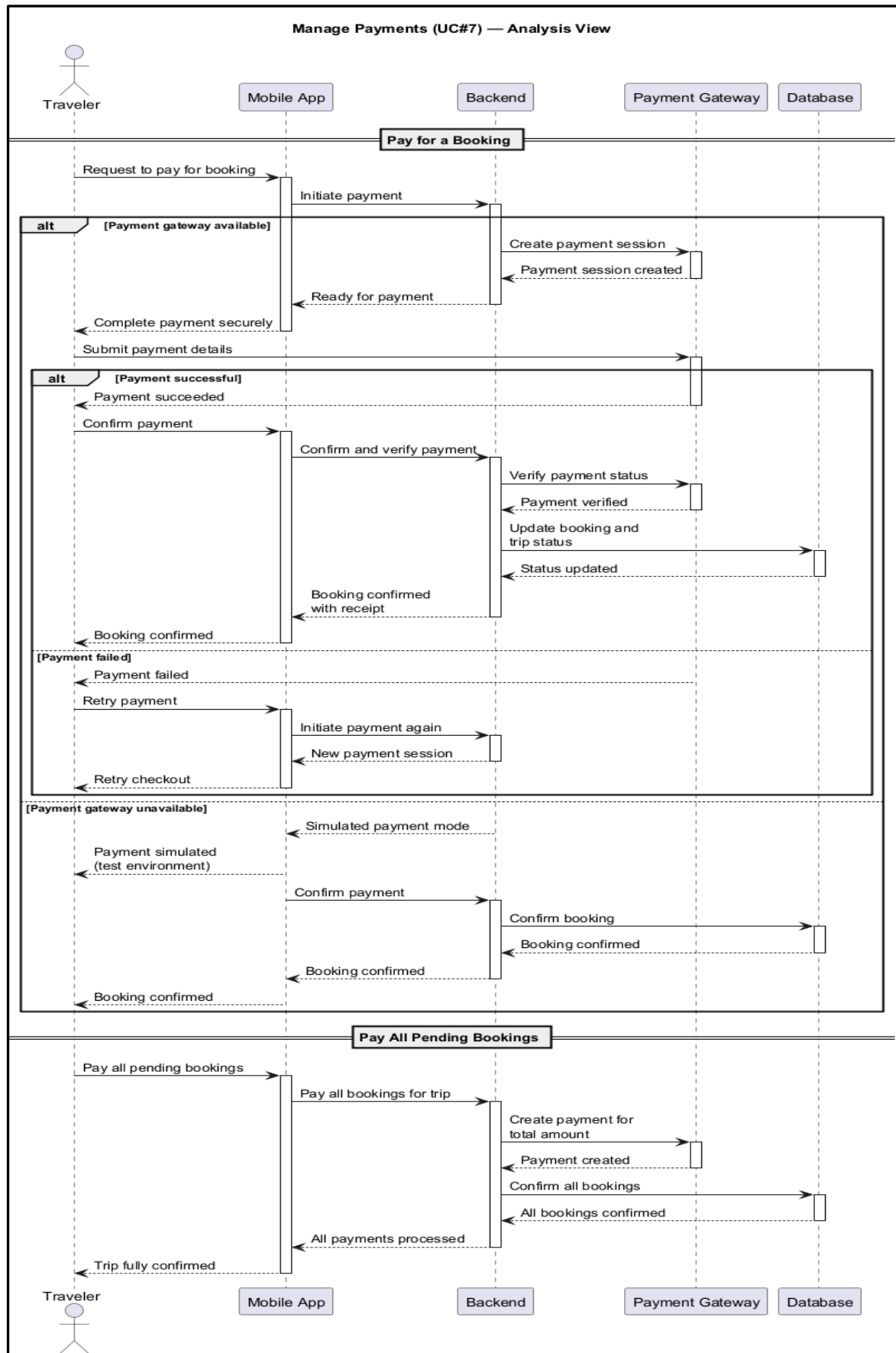


Figure 4.3.6 Manage Payments Sequence diagram

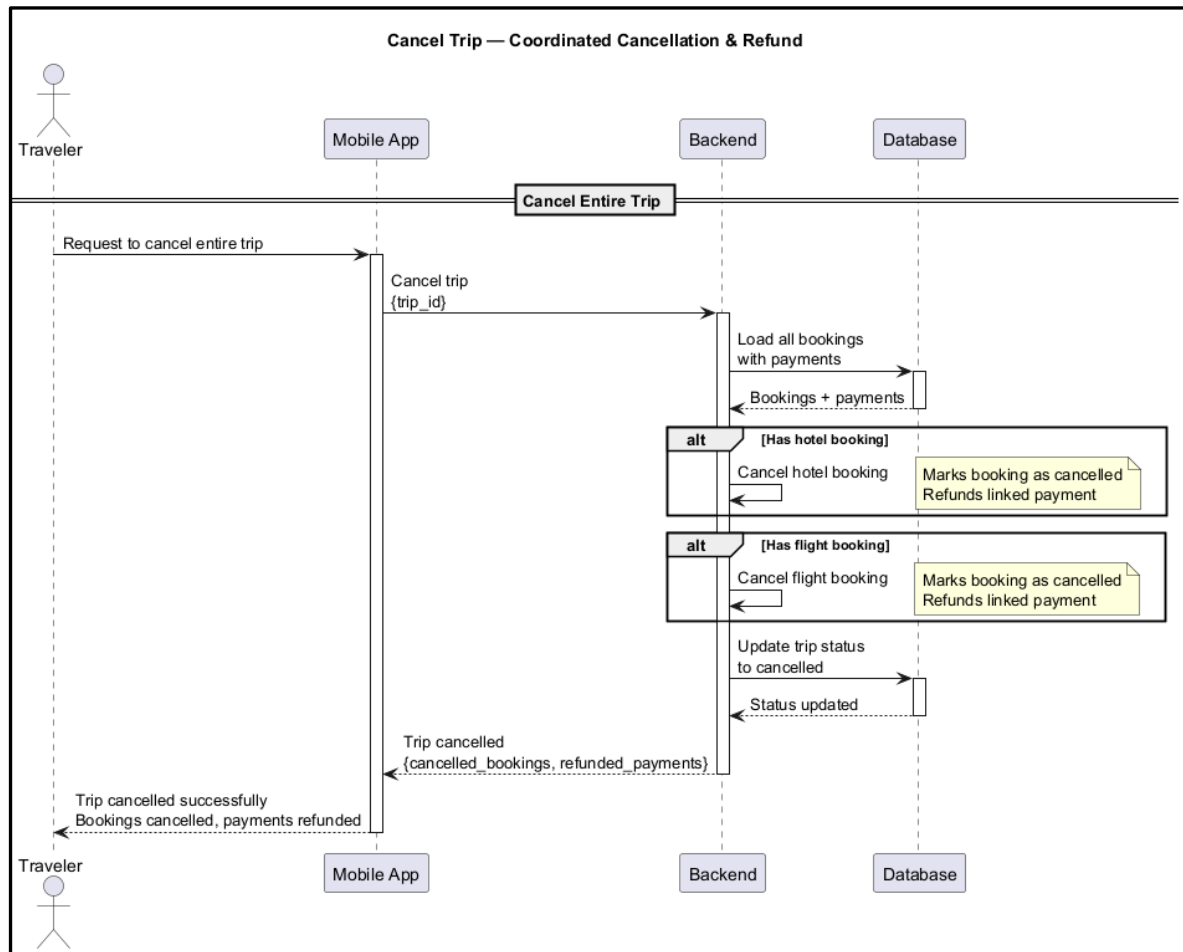
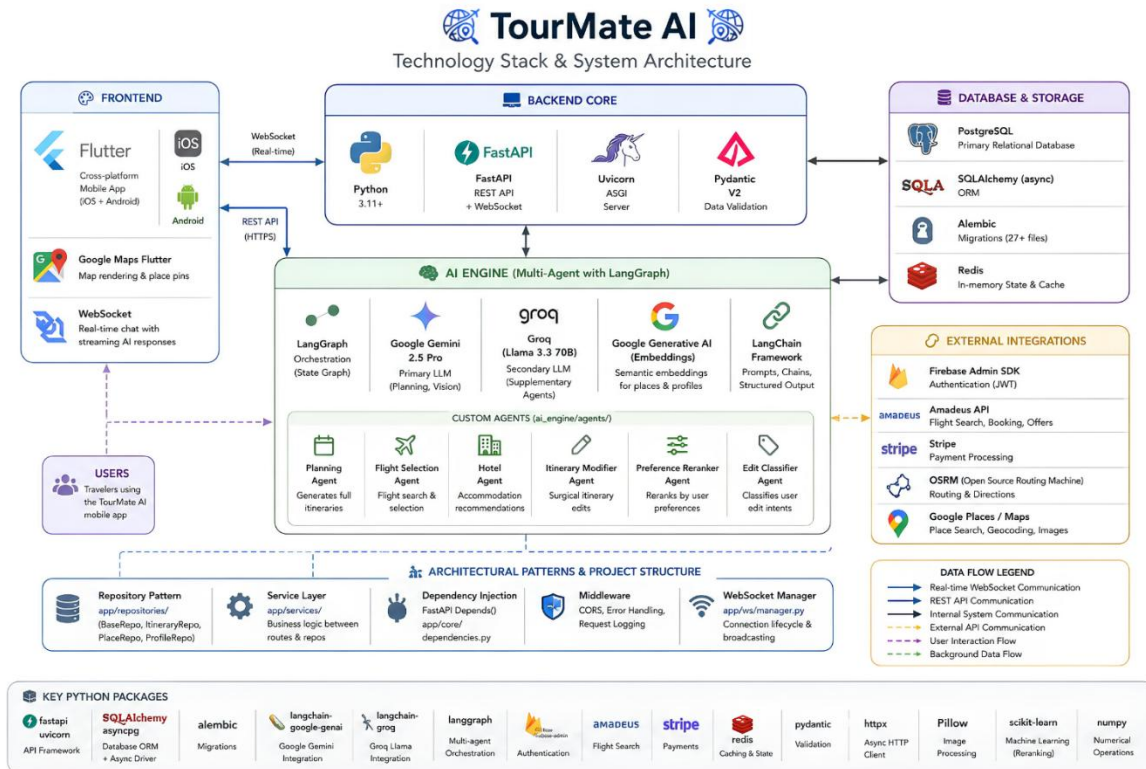


Figure 4.3.7 Coordinated Cancellation & Refund Sequence Diagram

4.4 Tools and Technologies Used

TourMate AI leverages a modern, AI-centric technology stack designed for autonomous decision-making, scalability, and cross-platform interaction. Each component was selected to ensure efficient integration of agentic reasoning, multimodal processing, and real-time itinerary optimization.



4.4.1 Core Development Frameworks

- FastAPI (Python):** Chosen for its high performance and native support for asynchronous workflows. It efficiently handles concurrent AI agent executions and real-time data processing. The backend uses async SQLAlchemy with asyncpg for non-blocking database operations, and Alembic for schema migration management across 30 migration versions.
- Flutter:** Powers the mobile application, providing a high-performance, cross-platform experience for both Android and iOS users from a single codebase. The mobile app follows a feature-first architecture with flutter_bloc for state management, freezed for immutable data classes, and Retrofit for type-safe API communication.

4.4.2 AI Orchestration and Vision

- TourMate AI is organized into two cooperating layers. The Conversation Orchestrator is a phase-driven state machine that manages the dialogue and, at the appropriate phase, invokes the flight-selection, hotel-selection, preference-reranking, and itinerary-modification agents. Nested within it, the Itinerary Generation Pipeline is a LangGraph graph of six nodes that runs during plan generation and produces a validated itinerary through a shared, persistent state (TripState):

1. **Load Profile** retrieves the per-trip profile from PostgreSQL
2. **Place Retriever** filters candidates using SQL-style criteria (city, category, rating, price level)
3. **Candidate Scorer** ranks candidates with a multi-signal formula combining popularity, semantic similarity, geographic proximity, rating, and diversity
4. **Planning Agent** generates the day-by-day itinerary with Gemini 2.5 Flash
5. **Route Optimizer** reorders stops using OSRM travel-time data with nearest-neighbour initialization and 2-opt local search
6. **Validation Agent** applies programmatic feasibility rules plus an LLM quality check with Groq Llama 3.3 70B, looping back to the Planning Agent for up to two retries when validation fails

Preference Reranker and **Hotel Agent** are deliberately kept outside this graph, they are executed by the orchestrator after the user requests a preference shift or approves the itinerary, respectively, so that itinerary edits and validation retry do not trigger redundant hotel selections.

- **Google Gemini 2.5 Flash (LLM + VLM):** Performs dual roles — as a reasoning model for itinerary generation (planning agent), and as a Vision-Language Model for processing user-uploaded travel images to extract style and preference signals.
- **Groq Llama 3.3 70B Versatile:** Powers the Unified Router, a single context-aware LLM call that replaces the previous three-call pattern (intent parser + clarification + general chat). The router uses Pydantic structured output to guarantee valid JSON responses, handling intent classification, slot extraction, and response generation in one shot.
- **Unified Router:** A single structured-output LLM call that sees full conversation history and current state (collected slots, phase, itinerary). It extracts travel information, decides the next action (`plan_trip`, `ask_clarification`, `answer_question`, `approve_itinerary`, `modify_itinerary`), and generates the response — all in one call. This replaces the previous three-call pattern and reduces latency by 60%.
- **Multi-Provider LLM Configuration:** A centralized registry within the AI engine's LLM configuration module maps each agent role to a specific provider and model. Both Google Gemini and Groq support multi-key rotation — when a key hits a rate limit (429 error) or a transient failure occurs, the system automatically switches to the next available key or retries with backoff. This enables staying within free-tier limits while maintaining reliability.

4.4.3 Data Management

- **PostgreSQL:** Stores user profiles, trip itineraries, place data (POIs with Bayesian popularity scores), reviews, saved places, recommendations, and conversation history. Uses async SQLAlchemy with the asyncpg driver for non-blocking operations.
- **Redis:** Manages conversation session state with automatic TTL (1 hour). Each user session tracks the conversation phase (`GREETING` → `SLOT_FILLING` → `PLAN_GENERATION` → `ITINERARY_REVIEW` →

FLIGHT_SELECTION → HOTEL_SELECTION → BOOKING → COMPLETED), accumulated slot data, message history, and itinerary state.

- **Firestore Authentication:** Manages user registration, login, and session handling via email and password credentials, integrating seamlessly with Flutter via the FlutterFire SDK. Firestore Authentication also provides native support for social sign-in providers such as Google, which the architecture can extend to as the application evolves.
- **Image Storage:** User-uploaded images submitted for VLM-based preference inference are stored directly in PostgreSQL and passed to the Vision-Language Model as inline data for analysis, removing the need for a separate media storage service.

4.4.4 External Services and Integration

The system integrates with several external services and supporting tools to enhance itinerary generation, route optimization, and system observability.

- **OSRM (Open-Source Routing Machine):** Provides real-world routing and travel-time matrices between itinerary locations. The itinerary optimization component utilizes OSRM travel-time data in conjunction with a 2-opt local search algorithm to reorder itinerary stops and minimize total travel time while maintaining itinerary feasibility.
- **Pre-Seeded Place Datasets:** Tourist attractions, restaurants, and hotels for each supported city (Cairo, Alexandria, Luxor, Aswan) are sourced from curated, pre-seeded datasets loaded into PostgreSQL, serving as the primary source of place data used throughout the recommendation and itinerary generation processes.
- **LangSmith:** Provides observability and debugging capabilities through the `@traced` decorator, enabling end-to-end tracing of agent execution. The trace hierarchy captures the complete workflow, including profile loading, preference analysis, retrieval, ranking, planning, optimization, and validation stages, facilitating monitoring and troubleshooting during development.
- **Token Usage Tracking:** A built-in TokenTracker component monitors prompt and completion token consumption for every Large Language Model (LLM) invocation across supported providers. The tracker supports per-role

aggregation and generates formatted usage summaries to assist with performance analysis and cost monitoring.

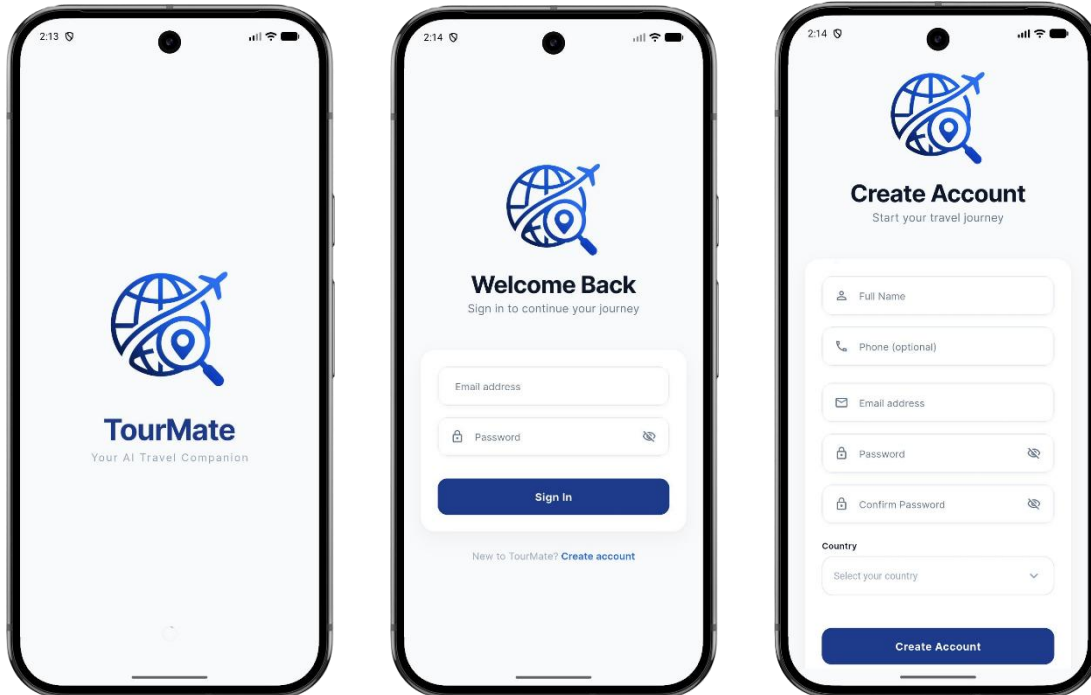
To ensure a safe academic environment, hotel booking is implemented in full simulation mode, flight booking integrates with the Amadeus test environment to reflect real airline data without finalizing an actual reservation, and payment processing runs through a sandbox environment with automatic fallback to a fully simulated payment when a live provider is not configured.

4.4.5 Development and Design Tools

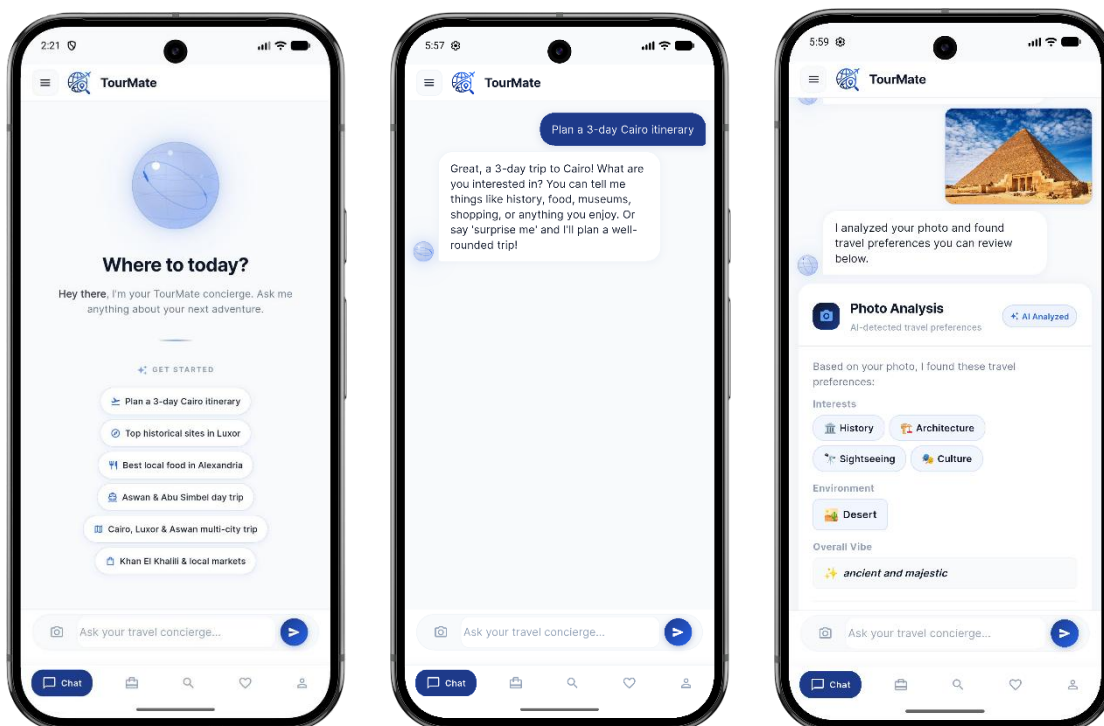
- **Visual Studio Code:** Served as the primary IDE for both backend and Flutter development.
- **PyCharm:** Used for backend development, particularly for Python-specific features such as intelligent code analysis, debugging, refactoring, and test execution.
- **Android Studio:** Used for Flutter application debugging, Android emulator management, device testing, and Android-specific build configuration.
- **pgAdmin 4:** Used to manage the PostgreSQL database, including database creation, schema management, query execution, and data inspection during development and testing.
- **Jira:** Used for project management and sprint planning, supporting the Agile phase of the hybrid methodology through task tracking, backlog management, and sprint organization.
- **Figma:** Used for UI/UX design, wireframing, and prototyping of all application screens before implementation.
- **Canva:** Used to design the project presentation slides, providing a visually consistent and professional format for academic delivery.
- **Lucidchart:** Used to design UML diagrams, agent interaction flows, and architecture diagrams.
- **Microsoft Word:** Employed for technical documentation and project reporting.

4.5 Application Screens (UI)

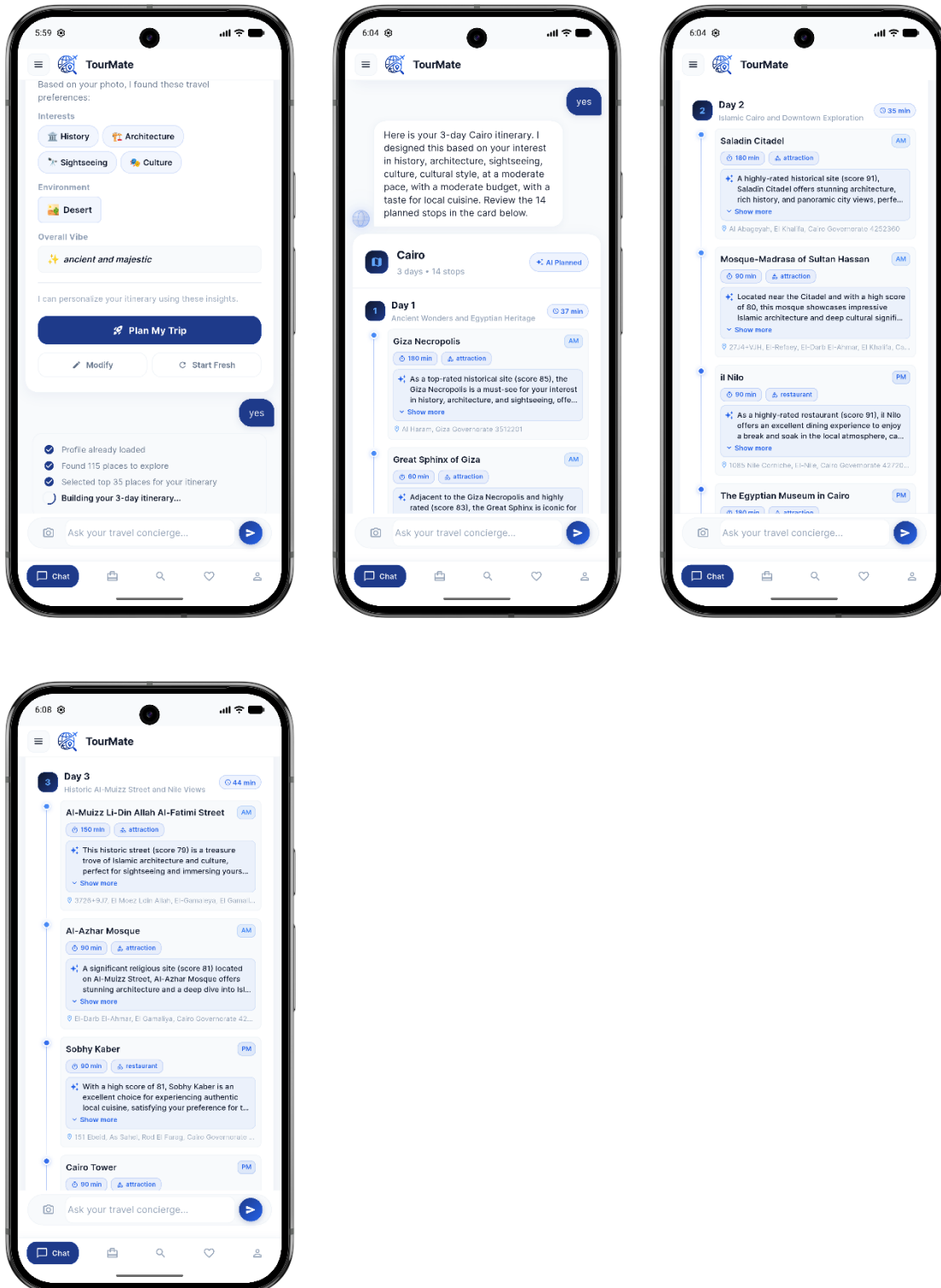
User Authentication Flow - Splash Screen, Sign In, and Sign Up with Email Credentials or Social Media Integration



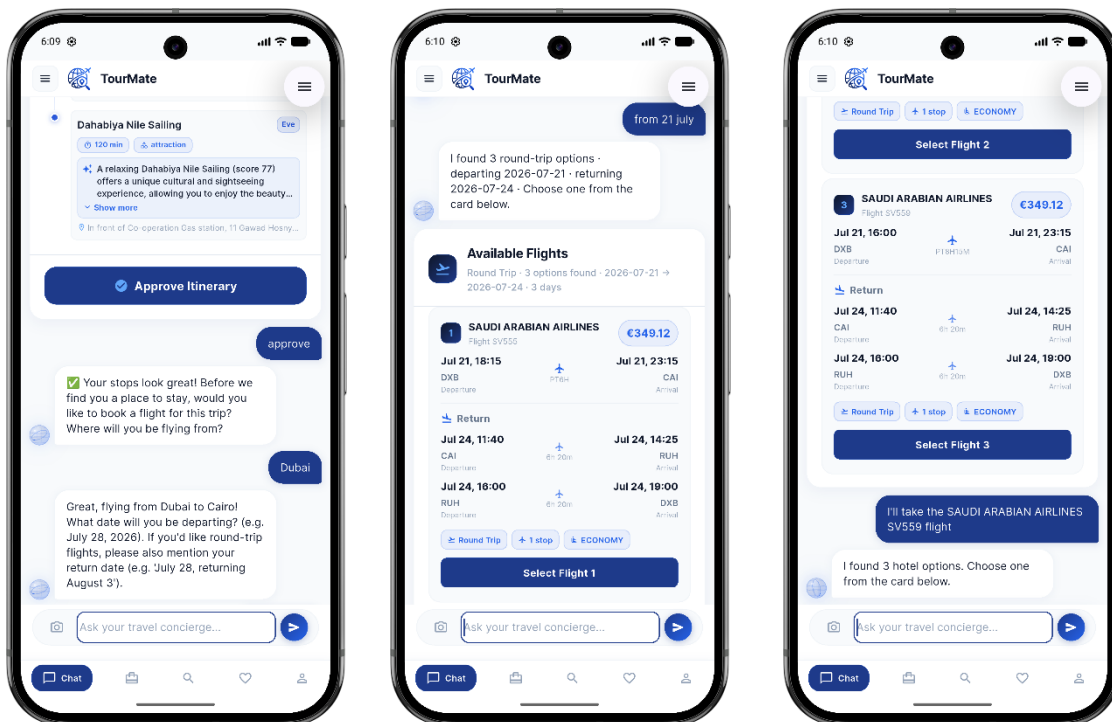
AI-Powered Personalized Travel Planning Workflow – Describe Your Preferences, Generate a Multi-Day Itinerary, Review Daily Recommendations, and Approve.



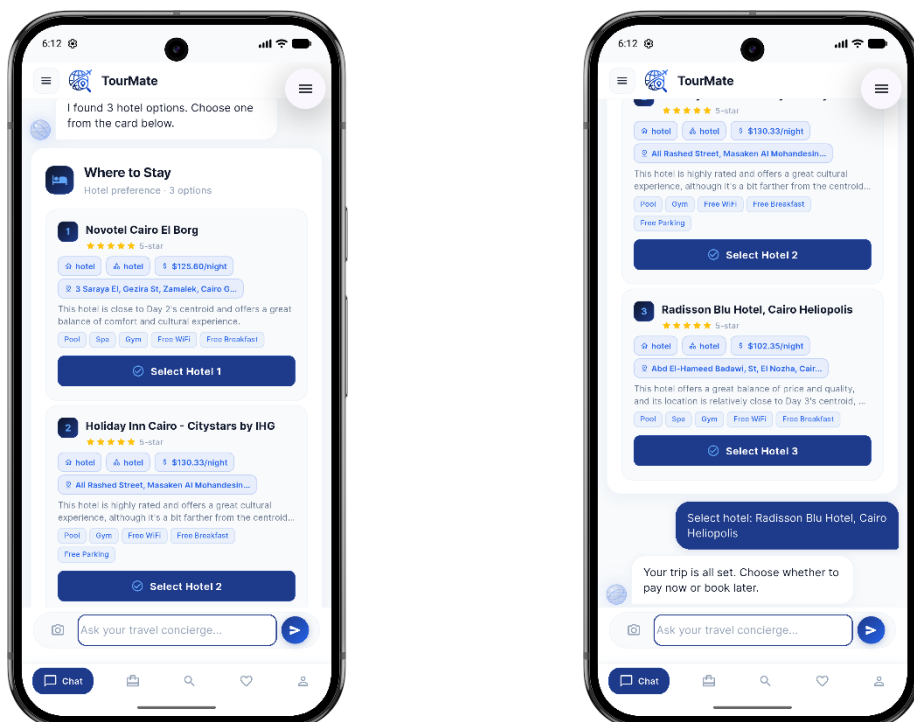
Chapter (4) TourMate AI Design and Implementation



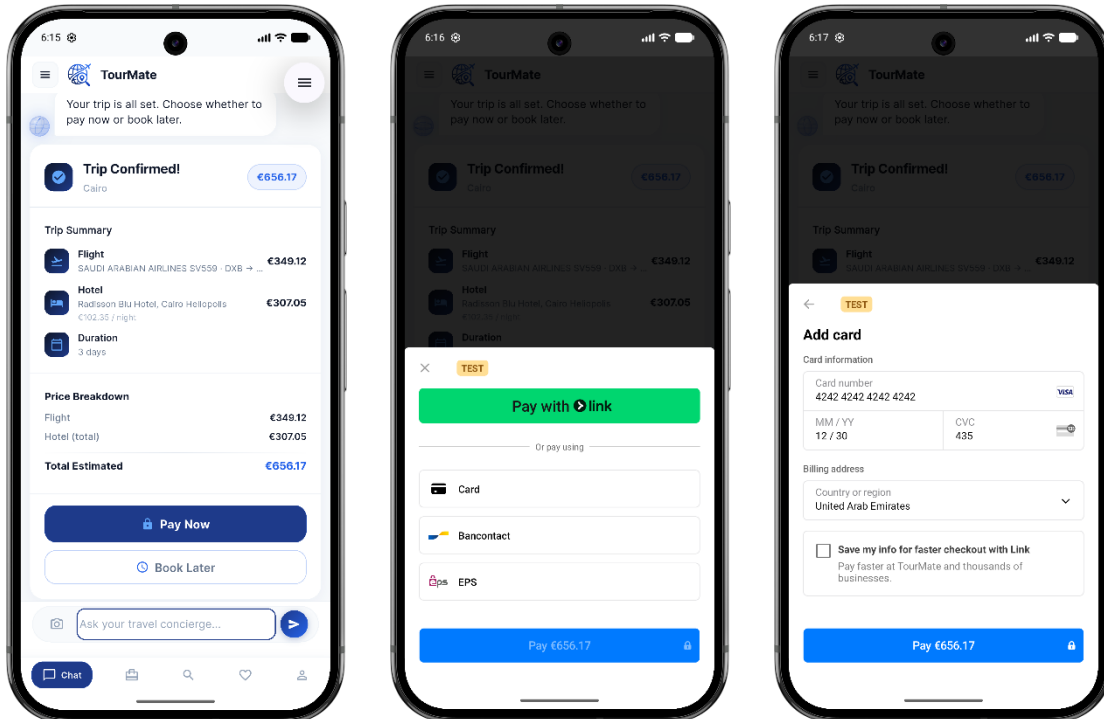
AI-Assisted Travel Booking Workflow – Browse Available Flights, Select a Preferred Flight, and Explore Personalized Hotel Recommendations.



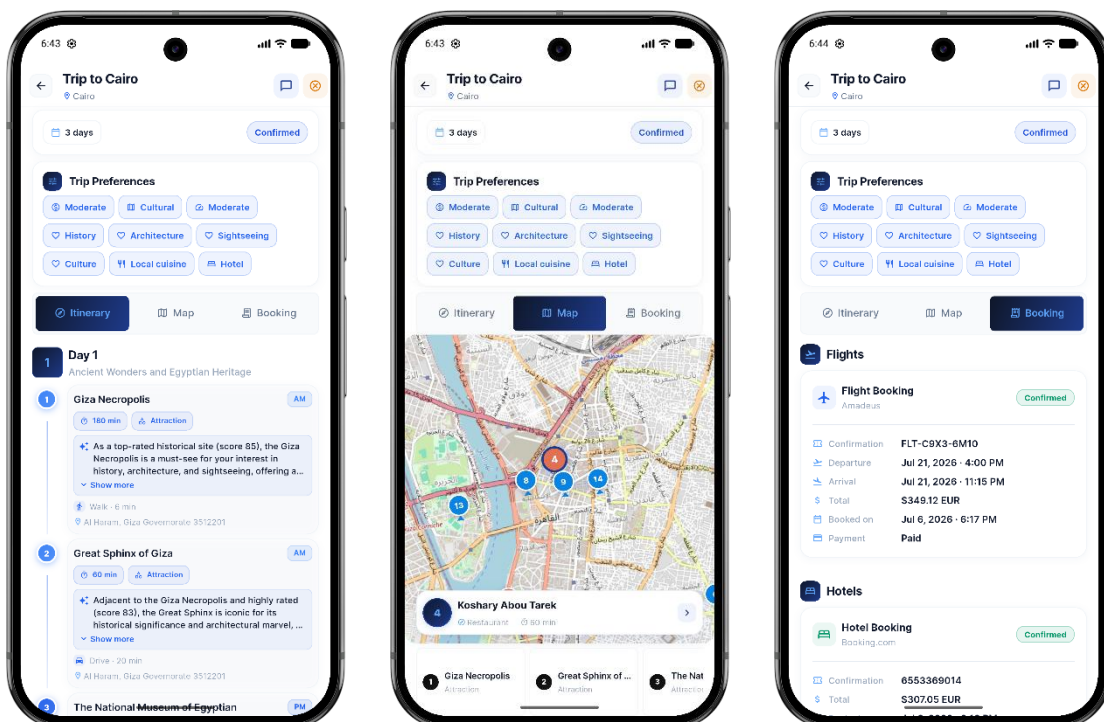
AI-Assisted Travel Booking Workflow – Browse Available Hotels, Select a Preferred Hotel, and Confirm Your Accommodation.



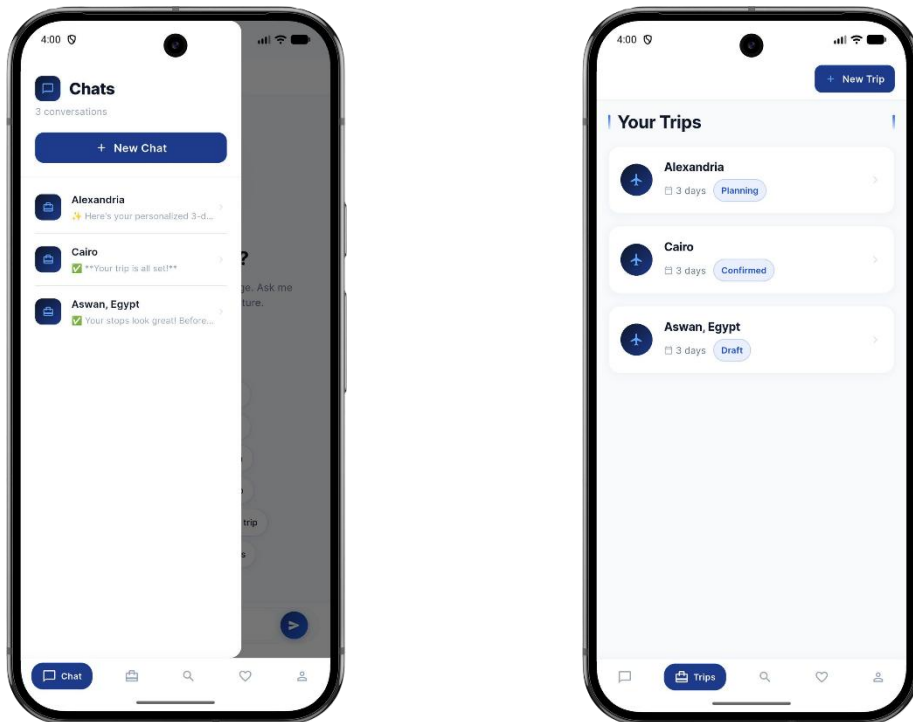
AI-Assisted Flight Booking Workflow – Review the Proposed Itinerary, Browse Available Flights, and Select Your Preferred Flight.



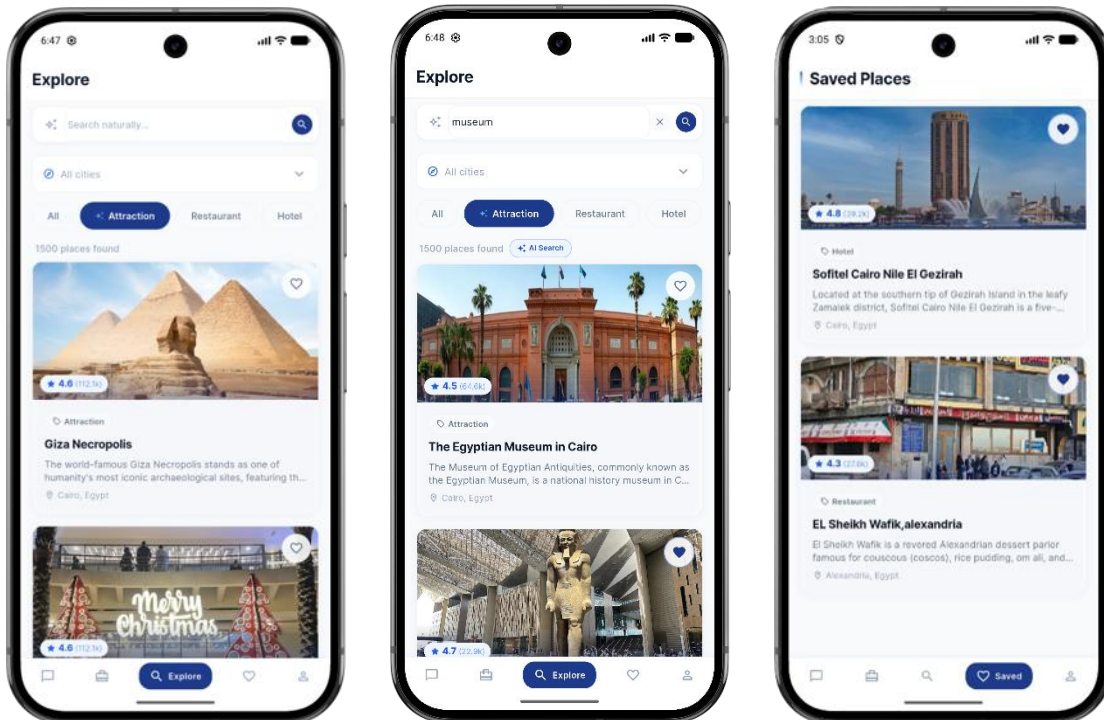
Interactive Itinerary Visualization – Explore AI-Generated Travel Plans Through Detailed Itinerary and Interactive Map Views.



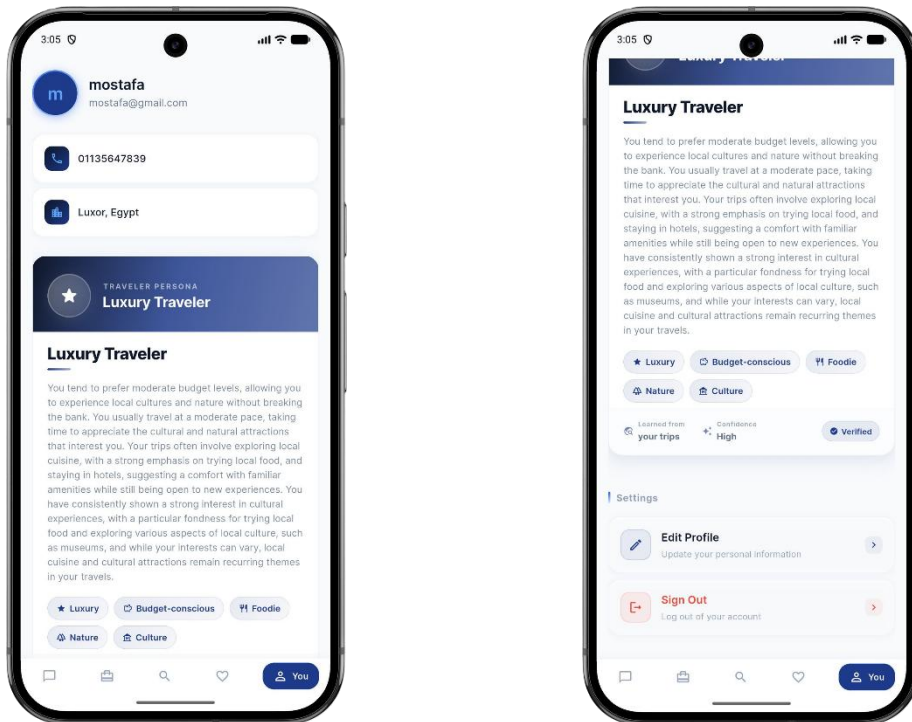
Trip Management Interface – Access Previous Conversations and Manage Saved Trips Throughout the Travel Planning Lifecycle.



Destination Discovery Interface – Explore Attractions, Hotels, and Restaurants, and Save Favorite Places for Future Trips.



AI-Generated Traveler Profile – View Personalized Travel Preferences and an Automatically Generated Traveler Persona.



Chapter (5) TourMate AI Testing and Evaluation

5.1 Testing Environment

All backend features were tested using pytest and FastAPI TestClient with Postman for manual verification. AI agent behavior (LangGraph) was validated through controlled prompt scenarios. The mobile app uses flutter_test and mockito for unit tests, with manual UI verification on the Android Emulator.

Table 5.1.1 Testing Types and Tools Table

Layer	Testing Type	Tools Used
Backend API (FastAPI)	Unit & Integration	pytest, FastAPI TestClient
Backend API (FastAPI)	Manual	Postman, curl
AI Agent Layer (LangGraph)	Agent Behavior	pytest, controlled prompt scenarios
AI Vision (Gemini VLM)	Multimodal Input	Manual via Postman and UI
Database (PostgreSQL)	Data Integrity	SQLAlchemy + asyncpg + alembic
Authentication (Firebase)	Auth Flow	firebase-admin SDK (mocked)
Frontend (Flutter)	Unit & Manual UI	flutter_test, mockito, Android Emulator

5.2 Test Cases

Category 1: Authentication & Account Management

Table 5.2.1 User Registration via Email Test Case

Test Case ID	TC-001
Test Case Name	User Registration via Email
Endpoint	POST /api/auth/register
Preconditions	The user is not already registered. The backend server and database are running.
Input Data	Email, Password (via Firebase), Full Name, Phone Number, Home City
Objective	Verify that a new user can register using an email address and password.
Expected Result	HTTP 201 Created is returned and a new user profile is initialized with provided details.
Actual Result	The registration request returned HTTP 201 Created, and the user profile was successfully created in the database.
Status	Passed

Table 5.2.2 User Registration with Existing Email Test Case

Test Case ID	TC-002
Test Case Name	User Registration with Existing Email
Endpoint	POST /api/auth/register
Preconditions	The user is already registered with the given Firebase UID.
Input Data	Existing Firebase token with already-registered user
Objective	Verify that registering with an already-existing account returns a conflict error.
Expected Result	HTTP 409 Conflict with "User already exists" message.
Actual Result	The request returned HTTP 409 with the appropriate error message.
Status	Passed

Table 5.2.3 User Login (Auto-Registration) Test Case

Test Case ID	TC-003
Test Case Name	User Login (Auto-Registration)
Endpoint	POST /api/auth/login
Preconditions	The backend server and Firebase are running. The user may or may not exist in the database.
Input Data	Valid Firebase ID token
Objective	Verify that a user can log in. If the user does not exist, a new user record is auto-created.
Expected Result	HTTP 200 OK with user data. A new user is created if they did not exist before.
Actual Result	The login request returned HTTP 200, and the user was either found or created successfully.
Status	Passed

Table 5.2.4 Unauthenticated Access to Protected Endpoint Test Case

Test Case ID	TC-004
Test Case Name	Unauthenticated Access to Protected Endpoint
Endpoint	Various protected endpoints (e.g., GET /trips/)
Preconditions	Backend server is running. No auth token is provided.
Input Data	None (no Firebase token)
Objective	Verify that protected endpoints return 401 Unauthorized when no valid token is provided.
Expected Result	HTTP 401 Unauthorized for all protected endpoints.
Actual Result	The request returned HTTP 401 with an authentication error.
Status	Passed

Table 5.2.5 Expired or Invalid Firebase Token Test Case

Test Case ID	TC-005
Test Case Name	Expired or Invalid Firebase Token
Endpoint	Various protected endpoints
Preconditions	Firebase token is expired or invalid.
Input Data	Expired Firebase ID token
Objective	Verify that expired/invalid Firebase tokens are rejected with 401 Unauthorized.
Expected Result	HTTP 401 Unauthorized with authentication error.
Actual Result	The request returned HTTP 401.
Status	Passed

Category 2: Intelligent Trip Profiling & Personalization

Table 5.2.6 Trip Profile Created After Status Update to Active Test Case

Test Case ID	TC-006
Test Case Name	Trip Profile Created After Status Update to Active
Endpoint	PATCH /trips/{trip_id}/status + GET /trips/{trip_id}/profile
Preconditions	User is authenticated. Trip exists with AI-generated profile data in the pipeline result.
Input Data	Status update to "active"
Objective	Verify that a TripProfile is created/updated with AI-generated preferences (budget, travel style, pace, interests, food, accommodation) when the trip status is updated to "active".
Expected Result	TripProfile exists with all enum fields correctly mapped and list fields populated.
Actual Result	TripProfile was created with correct values for all fields.
Status	Passed

Table 5.2.7 Traveler Persona Updated After Trip Approval Test Case

Test Case ID	TC-007
Test Case Name	Traveler Persona Updated After Trip Approval
Endpoint	(Internal – triggered by trip approval via chat)
Preconditions	Trip is approved. Trip profile has preference data (budget, style, pace, interests).
Input Data	Approved trip with trip profile
Objective	Verify that the traveler persona is updated based on the approved trip's preferences, merging with existing persona and preference counts.
Expected Result	User's traveler_persona is updated with new insights. Preference counts are incremented.
Actual Result	Traveler persona was successfully updated with merged preferences.
Status	Passed

Table 5.2.8 Preference-Based Place Reranking Test Case

Test Case ID	TC-008
Test Case Name	Preference-Based Place Reranking
Endpoint	(Internal AI Engine – Mode 1)
Preconditions	Trip has a profile with preferences. Candidate pool has diverse places.
Input Data	Candidate pool + trip profile (budget, interests, style, pace)
Objective	Verify that places are reranked according to the user's preferences, with preference-matched items scoring higher.
Expected Result	Places matching user preferences receive higher composite scores and are prioritized in results.
Actual Result	Preference reranking correctly boosted matching places.
Status	Passed

Category 3: Conversational & Multimodal Trip Requests

Table 5.2.9 WebSocket New Chat Connection Test Case

Test Case ID	TC-009
Test Case Name	WebSocket New Chat Connection
Endpoint	WS /ws/chat/new?token={token}
Preconditions	Valid Firebase token. Backend server, database, and Redis are running.
Input Data	WebSocket connection with valid token query parameter
Objective	Verify that a new WebSocket chat connection can be established with valid authentication.
Expected Result	WebSocket connection accepted, ping/pong heartbeat works, and chat messages can be exchanged.
Actual Result	WebSocket connection established successfully with bidirectional communication.
Status	Passed

Table 5.2.10 Full AI Planning Pipeline via WebSocket Test Case

Test Case ID	TC-010
Test Case Name	Full AI Planning Pipeline via WebSocket
Endpoint	WS /ws/chat/new?token={token}
Preconditions	Valid token. AI engine, Amadeus API, Redis, and database are operational.
Input Data	User message: "Plan a 5-day trip to Cairo, Egypt for 2 people"
Objective	Verify the complete AI planning flow: slot filling extracts trip requirements → AI itinerary generation → profile creation → trip persistence in the database → real-time streaming of results to the client.
Expected Result	Streaming response tokens → progress events → itinerary_data with days/stops → trip_created with trip_id → trip_approved.
Actual Result	Full pipeline completed successfully: slots extracted, itinerary generated, trip created in DB, itinerary data sent to client.
Status	Passed

Table 5.2.11 Image Upload in Chat Test Case

Test Case ID	TC-011
Test Case Name	Image Upload in Chat
Endpoint	WS /ws/chat/{trip_id}?token={token}
Preconditions	Trip exists. WebSocket is connected.
Input Data	JSON with "image" field containing base64-encoded image data
Objective	Verify that users can upload images in chat and the AI processes them for travel inspiration.
Expected Result	Image is decoded and passed to the AI engine. Vision features are extracted and persisted.
Actual Result	Image was processed successfully and vision features were extracted.
Status	Passed

Table 5.2.12 LLM API Timeout (AI Pipeline Failure) Test Case

Test Case ID	TC-012
Test Case Name	LLM API Timeout – AI Pipeline Failure
Endpoint	WS /ws/chat/new?token={token} (AI planning pipeline)
Preconditions	AI engine is operational. LLM provider experiences high latency (> 10s response time) or temporary unavailability.
Input Data	User message: "Plan a 5-day trip to Cairo, Egypt for 2 people" when LLM API is timing out
Objective	Verify that LLM API timeouts are handled gracefully — the pipeline retries with exponential backoff instead of crashing, and a fallback message is returned if all retries fail.
Expected Result	Pipeline retries up to 3 times with backoff (100ms, 200ms, 400ms). If all retries fail, a graceful error message is returned. No unhandled exceptions propagate to the WebSocket.

Actual Result	Initial test showed the LLM timeout propagating as an unhandled Exception, crashing the entire planning pipeline and terminating the user's session with no error message. After adding retry logic, timeouts are handled with automatic retries and fallback messaging works correctly.
Status	Failed
Defect Found	Yes
Defect Description	• The AI pipeline made only one attempt to call the LLM, with no retry logic — any timeout or temporary error crashed the entire planning pipeline. • The embedding call also ran synchronously, blocking the server from handling other users' requests at the same time.
Fix Applied	• Added automatic retries with increasing wait times (100ms, 200ms, 400ms) for temporary errors. • Switched the embedding call to a non-blocking (async) version so it no longer freezes other requests. • Added a friendly fallback message instead of crashing when all retries fail.

Category 4: AI-Powered Itinerary Generation & Editing

Table 5.2.13 Follow-up Itinerary Modification Test Case

Test Case ID	TC-013
Test Case Name	Follow-up Itinerary Modification
Endpoint	WS /ws/chat/{trip_id}?token={token}
Preconditions	Trip exists in "itinerary_draft" status. WebSocket is connected.
Input Data	User message: "Add a visit to the Egyptian Museum on Day 2"
Objective	Verify that follow-up modification requests are processed and the itinerary is updated with new stops.
Expected Result	AI processes the modification request, updates itinerary with new stop, and sends itinerary_updated + actions.
Actual Result	The modification was applied: new stop added to Day 2, itinerary_updated event sent.
Status	Passed

Table 5.2.14 Itinerary Route Optimization Test Case

Test Case ID	TC-014
Test Case Name	Itinerary Route Optimization
Endpoint	(Internal AI Engine)
Preconditions	Itinerary has multiple stops in a day with coordinates.
Input Data	List of stops with lat/lng coordinates
Objective	Verify that the route optimizer reorders stops by proximity to minimize travel time, preserving required anchor points.
Expected Result	Stops are reordered in an efficient route order. Travel times are annotated.
Actual Result	Route optimization correctly reordered stops and annotated travel times.
Status	Passed

Table 5.2.15 Itinerary Validation Checks

Test Case ID	TC-015
Test Case Name	Itinerary Validation Checks
Endpoint	(Internal AI Engine – ItineraryValidator)
Preconditions	Itinerary exists with days and stops.
Input Data	Generated itinerary structure
Objective	Verify that the itinerary validator performs programmatic checks (day ordering, stop ordering, timing feasibility, category diversity, geographic coverage).
Expected Result	Validation results with pass/fail for each check and explanatory messages.
Actual Result	Itinerary validation ran successfully with comprehensive check results.
Status	Passed

Table 5.2.16 Edit Intent Classification Test Case

Test Case ID	TC-016
Test Case Name	Edit Intent Classification
Endpoint	(Internal AI Engine – EditClassifier)
Preconditions	AI engine is operational.
Input Data	User messages with modification intents (e.g., "add restaurant on Day 2", "swap the museum and the park")
Objective	Verify that the edit classifier correctly routes messages to the appropriate itinerary operations (ADD, REMOVE, SWAP, REORDER, CHANGE_HOTEL).
Expected Result	Modification requests are correctly classified to the right operation type.
Actual Result	Edit classification worked correctly for all operation types.
Status	Passed

Table 5.2.17 LLM Malformed JSON (Itinerary Parsing Crash) Test Case

Test Case ID	TC-017
Test Case Name	LLM Malformed JSON – Itinerary Parsing Crash
Endpoint	Internal AI Engine – ItineraryValidator (ai_engine/services/itinerary_validator.py)
Preconditions	AI engine is operational. LLM is generating itinerary data.
Input Data	LLM returns itinerary data with malformed JSON (missing braces, trailing commas, truncated output, markdown code fences)
Objective	Verify that the itinerary validator handles malformed JSON from the LLM gracefully — repairing common structural issues and retrying before failing.
Expected Result	Malformed JSON is repaired using structural analysis (find outermost {}, auto-close braces, trim markdown). If repair fails, the LLM is retried with parsing error feedback (up to 3 attempts). If all retries fail, a structured validation failure is returned without crashing.
Actual Result	Initial test showed LLM outputs with unclosed braces, trailing commas, and markdown code fences causing json.loads() to raise an unhandled json.JSONDecodeError, aborting the entire itinerary generation. After adding JSON repair utilities, >90% of malformed LLM outputs are successfully repaired and parsed.
Status	Failed
Defect Found	Yes
Defect Description	• LLM outputs are probabilistic and occasionally break the expected JSON schema: missing braces, trailing commas, markdown code fences, or truncated output. • The original code could not recover from these structural issues and crashed instead of continuing.

Fix Applied

- Added JSON repair logic: finds the outermost {} block, discards surrounding text, and auto-closes unmatched brackets/braces.
- Removes trailing commas and strips markdown code fences before parsing.
- Added a retry loop (up to 3 attempts) that sends the parsing error back to the LLM for a corrected response.
- If all retries fail, the validator returns a structured failure instead of crashing.

Category 5: Place Discovery & Exploration

Table 5.2.18 Explore Places (Default to Egypt) Test Case

Test Case ID	TC-018
Test Case Name	Explore Places – Default to Egypt
Endpoint	GET /places/explore
Preconditions	Database is seeded with places in Egypt and other countries.
Input Data	No parameters
Objective	Verify that exploring places with no filters defaults to Egypt and returns places sorted by popularity.
Expected Result	HTTP 200 OK with places from Egypt only, sorted by popularity_score descending.
Actual Result	The request returned HTTP 200 with Egypt places sorted by popularity.
Status	Passed

Table 5.2.19 Explore Places Filtered by Category Test Case

Test Case ID	TC-019
Test Case Name	Explore Places Filtered by Category
Endpoint	GET /places/explore?category=restaurant
Preconditions	Database has restaurants and other place types in Egypt.
Input Data	category=restaurant
Objective	Verify that the category filter works correctly and returns only places of the specified category.
Expected Result	HTTP 200 OK with only restaurants returned.
Actual Result	The request returned HTTP 200 with only restaurant places.
Status	Passed

Table 5.2.20 Get Place Detail Test Case

Test Case ID	TC-020
Test Case Name	Get Place Detail
Endpoint	GET /places/{place_id}
Preconditions	Database has a place with the given ID, including category-specific details.
Input Data	Valid place_id
Objective	Verify that fetching a place returns full details including category-specific data (hotel_details, restaurant_details, attraction_details).
Expected Result	HTTP 200 OK with complete place data including category-specific fields.
Actual Result	The request returned HTTP 200 with full place details.
Status	Passed

Table 5.2.21 Semantic Search with Natural Language Test Case

Test Case ID	TC-021
Test Case Name	Semantic Search with Natural Language
Endpoint	POST /places/search/semantic
Preconditions	Database has places with embeddings.
Input Data	Query, City, Limit
Objective	Verify that semantic search returns places ranked by cosine similarity to the natural language query.
Expected Result	HTTP 200 OK with places sorted by similarity_score descending.
Actual Result	The request returned HTTP 200 with semantically relevant places.
Status	Passed

Category 6: Flight & Hotel Booking

Table 5.2.22 Smart Flight Search by City Names Test Case

Test Case ID	TC-022
Test Case Name	Smart Flight Search by City Names
Endpoint	POST /flights/smart-search
Preconditions	Amadeus API is operational.
Input Data	JSON with origin_city="Cairo", destination_city="Dubai", departure_date, return_date, adults=1
Objective	Verify that the smart search resolves city names to IATA codes and returns flight offers.
Expected Result	HTTP 200 OK with flight offers plus resolved origin/destination airport info.
Actual Result	The request returned HTTP 200 with flight offers and resolved airport details.
Status	Passed

Table 5.2.23 Initiate Flight Booking (Price + PaymentIntent) Test Case

Test Case ID	TC-023
Test Case Name	Initiate Flight Booking – Price + PaymentIntent
Endpoint	POST /flights/book/initiate
Preconditions	User is authenticated. Flight raw_offer is available from search results.
Input Data	JSON with raw_offer, trip_id
Objective	Verify that initiating a flight booking prices the offer via Amadeus, creates a Stripe PaymentIntent, and returns client_secret + priced_offer.
Expected Result	HTTP 200 OK with client_secret, payment_intent_id, priced_offer, airline info, flight details.
Actual Result	The request returned HTTP 200 with all expected fields.
Status	Passed

Table 5.2.24 Confirm Flight Booking After Payment Test Case

Test Case ID	TC-024
Test Case Name	Confirm Flight Booking After Payment
Endpoint	POST /flights/book/confirm
Preconditions	Stripe PaymentIntent succeeded. Priced offer from initiate step.
Input Data	JSON with payment_intent_id, priced_offer, trip_id, traveler details
Objective	Verify that confirming a flight booking verifies Stripe, books via Amadeus, and creates Booking + Payment + Receipt records.
Expected Result	HTTP 201 Created with full FlightBookingResponse including booking_id, status, airline, flight numbers, payment info.
Actual Result	The request returned HTTP 201 with flight booking confirmed.
Status	Passed

Table 5.2.25 Cancel Flight Booking Test Case

Test Case ID	TC-025
Test Case Name	Cancel Flight Booking
Endpoint	POST /flights/{booking_id}/cancel
Preconditions	Flight booking exists and belongs to user.
Input Data	Valid booking_id
Objective	Verify that cancelling a flight booking marks it as cancelled and refunds the associated payment.
Expected Result	HTTP 200 OK with FlightBookingResponse showing status="cancelled".
Actual Result	The request returned HTTP 200 with flight booking cancelled.
Status	Passed

Table 5.2.26 Hotel Selection with Preference Scoring Test Case

Test Case ID	TC-026
Test Case Name	Hotel Selection with Preference Scoring
Endpoint	(Internal AI Engine)
Preconditions	Trip has itinerary days with centroids. Candidate pool has hotel options.
Input Data	Trip profile + itinerary centroids + hotel candidates
Objective	Verify that the hotel selection algorithm scores hotels by preference match, proximity to daily centroids, and rating, returning the best option.
Expected Result	Hotels are scored and the top hotel(s) are selected based on composite scoring.
Actual Result	Hotel selection worked correctly with preference-aware scoring.
Status	Passed

Table 5.2.27 Amadeus API Rate Limiting (Throttling) Test Case

Test Case ID	TC-027
Test Case Name	Amadeus API Rate Limiting – Throttling
Endpoint	POST /flights/search (Amadeus API integration)
Preconditions	Amadeus API is configured. Multiple rapid flight search requests are sent within a short time window (> 3 requests/second).
Input Data	10 concurrent flight search requests with different origin/destination pairs
Objective	Verify that Amadeus API rate limiting is handled gracefully — the system does not crash when rate limits are exceeded and returns an appropriate, distinct error response to the user.
Expected Result	When Amadeus returns a 429 (Too Many Requests) response, the system detects this specific status code and returns HTTP 429 with a clear, user-friendly message. No unhandled exceptions propagate.

Actual Result	Amadeus API returns HTTP 429 when more than ~3 requests/second are sent. The client caught all exceptions generically and re-raised them as a plain ValueError, which the route converted to HTTP 400 with the raw Amadeus error text — not a distinct 429 response, and not a user-friendly message.
Status	Failed
Defect Found	Yes
Defect Description	• The Amadeus client's exception handling did not check the underlying HTTP status code, so 429 (rate limit) responses were treated the same as any other API error. • As a result, rate-limited requests returned a generic HTTP 400 with a raw technical message instead of a distinct, user-friendly response.
Fix Applied	• Added a dedicated AmadeusRateLimitError exception. • search_flights() now inspects the status code; if it's 429, it raises AmadeusRateLimitError with a clear message instead of a generic error. • The route now catches this separately and returns HTTP 429 with a friendly message, falling back to HTTP 400 only for other error types.
Retest Result	Rate-limited requests now return HTTP 429 with the message: "Flight search service is temporarily busy due to high demand. Please try again in a moment." Other Amadeus errors still return HTTP 400 as before. No unhandled 500 errors occur in either case.

Category 7: Payment Management

Table 5.2.28 Initiate Stripe Payment for Booking Test Case

Test Case ID	TC-028
Test Case Name	Initiate Stripe Payment for Booking
Endpoint	POST /bookings/{booking_id}/initiate-payment
Preconditions	Booking exists and belongs to user. Stripe sandbox is configured.
Input Data	JSON with payment_method and return_url
Objective	Verify that initiating payment creates a Stripe PaymentIntent and returns a client_secret for the Payment Sheet.
Expected Result	HTTP 200 OK with client_secret, stripe_payment_intent_id, payment_id. Trip status transitions to "payment_processing".
Actual Result	The request returned HTTP 200 with valid client_secret and PaymentIntent created.
Status	Passed

Table 5.2.29 Confirm Booking After Successful Payment Test Case

Test Case ID	TC-029
Test Case Name	Confirm Booking After Successful Payment
Endpoint	POST /bookings/{booking_id}/confirm-after-payment
Preconditions	PaymentIntent succeeded. Booking is in "pending" status with payment_record.
Input Data	JSON with stripe_payment_intent_id
Objective	Verify that the booking is confirmed after payment succeeds.
Expected Result	HTTP 200 OK with booking_id, status="confirmed" (or "already_confirmed").
Actual Result	The request returned HTTP 200 with booking confirmed.
Status	Passed

Table 5.2.30 Combined Flight & Hotel Booking Initiate Test Case

Test Case ID	TC-030
Test Case Name	Combined Flight + Hotel Booking Initiate
Endpoint	POST /bookings/combined/initiate
Preconditions	User is authenticated. Trip exists. Flight raw_offer and hotel place_id are available.
Input Data	JSON with trip_id, raw_offer, hotel_place_id, hotel_total_cost, hotel_start_datetime, hotel_end_datetime
Objective	Verify that a single Stripe PaymentIntent can cover both flight and hotel bookings combined.
Expected Result	HTTP 200 OK with combined client_secret, payment_intent_id, combined_amount, flight and hotel amounts.
Actual Result	The request returned HTTP 200 with combined PaymentIntent created.
Status	Passed

Table 5.2.31 Combined Booking Confirm After Payment Test Case

Test Case ID	TC-031
Test Case Name	Combined Booking Confirm After Payment
Endpoint	POST /bookings/combined/confirm
Preconditions	Stripe PaymentIntent succeeded. Combined initiate was called.
Input Data	JSON with trip_id, payment_intent_id, priced_offer, hotel_booking_id
Objective	Verify that both flight and hotel bookings are confirmed after combined Stripe success.
Expected Result	HTTP 200 OK with flight_booking_id, hotel_booking_id, status="confirmed". Trip status moves to "booking_confirmed".
Actual Result	The request returned HTTP 200 with both bookings confirmed.
Status	Passed

Table 5.2.32 Payment Failure (Booking Not Confirmed) Test Case

Test Case ID	TC-032
Test Case Name	Payment Failure – Booking Not Confirmed
Endpoint	POST /bookings/{booking_id}/initiate-payment (with Stripe failure)
Preconditions	Booking exists. Stripe PaymentIntent creation fails or payment fails.
Input Data	Valid booking_id
Objective	Verify that when payment initiation fails, the booking remains in "pending" status and the trip status reverts to "payment_failed" (recoverable).
Expected Result	HTTP 400 with error. Booking status remains "pending". Trip status set to "payment_failed".
Actual Result	The request returned HTTP 400 and trip status was set to "payment_failed".
Status	Passed

Category 8: Reviews & Personal Bookmarks

Table 5.2.33 Create Review for a Place Test Case

Test Case ID	TC-033
Test Case Name	Create Review for a Place
Endpoint	POST /reviews/
Preconditions	User is authenticated. Place exists. User has not reviewed this place before.
Input Data	JSON with place_id, rating (1-5), comment
Objective	Verify that a user can create a review with rating and comment for a place.
Expected Result	HTTP 201 Created with review_id, place_id, user_id, rating, comment, review_date.
Actual Result	The request returned HTTP 201 with review created.
Status	Passed

Table 5.2.34 Duplicate Review Returns Conflict Test Case

Test Case ID	TC-034
Test Case Name	Duplicate Review Returns Conflict
Endpoint	POST /reviews/
Preconditions	User has already reviewed this place.
Input Data	Same place_id as existing review
Objective	Verify that creating a second review for the same place returns 409 Conflict.
Expected Result	HTTP 409 Conflict with "You have already reviewed this place" message.
Actual Result	The request returned HTTP 409 with appropriate message.
Status	Passed

Table 5.2.35 Get Reviews for a Place with Summary Test Case

Test Case ID	TC-035
Test Case Name	Get Reviews for a Place with Summary
Endpoint	GET /reviews/place/{place_id}
Preconditions	Place has multiple reviews with varying ratings.
Input Data	Valid place_id
Objective	Verify that fetching place reviews returns all reviews with user names, average rating, total count, and likes info.
Expected Result	HTTP 200 OK with total_reviews, average_rating, and reviews array (each with user_name, rating, comment, likes_count, liked_by_user).
Actual Result	The request returned HTTP 200 with correct review data and summary.
Status	Passed

Table 5.2.36 Toggle Like on Review Test Case

Test Case ID	TC-036
Test Case Name	Toggle Like on Review
Endpoint	POST /reviews/{review_id}/like
Preconditions	User is authenticated. Review exists.
Input Data	Valid review_id (first call = like, second call = unlike)
Objective	Verify that toggling like increments/decrements the likes_count and creates/removes the ReviewLike record.
Expected Result	First call: likes_count increments by 1. Second call: likes_count decrements by 1.
Actual Result	Like/unlike toggle worked correctly.
Status	Passed

Table 5.2.37 Save a Place Test Case

Test Case ID	TC-037
Test Case Name	Save a Place
Endpoint	POST /saved-places/
Preconditions	User is authenticated. Place exists.
Input Data	JSON with place_id and optional note
Objective	Verify that a user can save a place (bookmark) with an optional note.
Expected Result	HTTP 201 Created with saved_place_id, place_id, saved_at timestamp.
Actual Result	The request returned HTTP 201 with place saved.
Status	Passed

Table 5.2.38 Duplicate Save Returns Conflict Test Case

Test Case ID	TC-038
Test Case Name	Duplicate Save Returns Conflict
Endpoint	POST /saved-places/
Preconditions	User has already saved this place.
Input Data	Same place_id as existing saved place
Objective	Verify that saving an already-saved place returns 409 Conflict.
Expected Result	HTTP 409 Conflict with "Place already saved" message.
Actual Result	The request returned HTTP 409 with appropriate message.
Status	Passed

Table 5.2.39 List Saved Places with Full Details Test Case

Test Case ID	TC-039
Test Case Name	List Saved Places with Full Details
Endpoint	GET /saved-places/
Preconditions	User has multiple saved places.
Input Data	None
Objective	Verify that listing saved places returns all bookmarked places with full place data embedded, ordered by most recently saved.
Expected Result	HTTP 200 OK with list of saved places, each containing saved_place_id, saved_at, and full place object with category-specific details.
Actual Result	The request returned HTTP 200 with saved places including full place details.
Status	Passed

Category 9: Image-Based Travel Inspiration

Table 5.2.40 Image Upload Triggers Vision Feature Extraction Test Case

Test Case ID	TC-040
Test Case Name	Image Upload Triggers Vision Feature Extraction
Endpoint	WS /ws/chat/{trip_id}?token={token}
Preconditions	Trip exists. WebSocket is connected.
Input Data	Base64-encoded travel image uploaded via chat
Objective	Verify that uploading an image triggers the AI vision pipeline to extract features (scene type, colors, objects, travel preferences).
Expected Result	Image features are extracted with has_signal=true. Image record is created in DB with extracted features.
Actual Result	Image features were extracted and persisted successfully.
Status	Passed

Table 5.2.41 Get Images for a Trip Test Case

Test Case ID	TC-041
Test Case Name	Get Images for a Trip
Endpoint	GET /images/trip/{trip_id}
Preconditions	Trip has uploaded images with extracted features.
Input Data	Valid trip_id
Objective	Verify that fetching images for a trip returns all images with their extracted vision features.
Expected Result	HTTP 200 OK with list of images including features, metadata, and timestamps.
Actual Result	The request returned HTTP 200 with images and features.
Status	Passed

Category 10: Trip Lifecycle Management

Table 5.2.42 Create Trip with Minimal Fields Test Case

Test Case ID	TC-042
Test Case Name	Create Trip with Minimal Fields
Endpoint	POST /trips/
Preconditions	User is authenticated. Backend server and database are running.
Input Data	Destination, Start Date, End Date
Objective	Verify that a minimal trip (destination + dates only) can be created successfully.
Expected Result	HTTP 201 Created with trip details, auto-generated itinerary with days, conversation, and first message.
Actual Result	The request returned HTTP 201 with trip data including itinerary, days, conversation_id, and auto_message.
Status	Passed

Table 5.2.43 Create Trip with All Fields Test Case

Test Case ID	TC-043
Test Case Name	Create Trip with All Fields
Endpoint	POST /trips/
Preconditions	User is authenticated. Backend server and database are running.
Input Data	Trip Name, Destination, Start Date, End Date, Number of Travelers
Objective	Verify that a trip with all optional fields can be created successfully.
Expected Result	HTTP 201 Created with complete trip details including all provided fields.
Actual Result	The request returned HTTP 201 with all provided fields correctly persisted.
Status	Passed

Table 5.2.44 List User Trips Test Case

Test Case ID	TC-044
Test Case Name	List User Trips
Endpoint	GET /trips/
Preconditions	User is authenticated and has at least one trip created.
Input Data	None
Objective	Verify that listing trips returns only the authenticated user's trips, ordered by most recent.
Expected Result	HTTP 200 OK with a list of trip summaries, each including trip_id, name, destination, dates, duration, status.
Actual Result	The request returned HTTP 200 with only the authenticated user's trips.
Status	Passed

Table 5.2.45 Get Single Trip Details Test Case

Test Case ID	TC-045
Test Case Name	Get Single Trip Details
Endpoint	GET /trips/{trip_id}
Preconditions	User is authenticated and owns the trip. The trip has an itinerary with days and stops.
Input Data	Valid trip_id
Objective	Verify that fetching a single trip returns full details including itinerary, days, stops, and pending booking count.
Expected Result	HTTP 200 OK with complete trip data including nested itinerary with days/stops.
Actual Result	The request returned HTTP 200 with full trip details including itinerary and pending bookings count.
Status	Passed

Table 5.2.46 Delete Trip and Cascaded Data Test Case

Test Case ID	TC-046
Test Case Name	Delete Trip and Cascaded Data
Endpoint	DELETE /trips/{trip_id}
Preconditions	User is authenticated and owns the trip. The trip has conversations, messages, itinerary, and bookings.
Input Data	Valid trip_id
Objective	Verify that deleting a trip removes all associated data (conversation, messages, itineraries, days, stops, bookings, profiles).
Expected Result	HTTP 200 OK with success message. All related records are cascade-deleted from the database.
Actual Result	The request returned HTTP 200 and all associated records were deleted successfully.
Status	Passed

Table 5.2.47 Update Trip Status Test Case

Test Case ID	TC-047
Test Case Name	Update Trip Status
Endpoint	PATCH /trips/{trip_id}/status
Preconditions	User is authenticated and owns the trip.
Input Data	New status value (e.g., "active", "itinerary_draft", "awaiting_booking")
Objective	Verify that a trip's status can be updated and that approval timestamps are set when transitioning to "active".
Expected Result	HTTP 200 OK with updated trip_id, status, and approved_at (if status=active).
Actual Result	The request returned HTTP 200 with status updated and approved_at timestamp set.
Status	Passed

Table 5.2.48 Cancel Trip with Bookings Test Case

Test Case ID	TC-048
Test Case Name	Cancel Trip with Bookings
Endpoint	POST /trips/{trip_id}/cancel
Preconditions	User is authenticated and owns the trip. Trip has associated hotel and/or flight bookings with payments.
Input Data	Valid trip_id
Objective	Verify that cancelling a trip cancels all associated bookings, refunds payments, and marks the trip as "cancelled".
Expected Result	HTTP 200 OK with list of cancelled booking IDs, refunded payment IDs, and trip status "cancelled".
Actual Result	The request returned HTTP 200 with all bookings cancelled and payments refunded.
Status	Passed

Category 11: Conversation Flow & Context Management

Table 5.2.49 Clear Chat History Test Case

Test Case ID	TC-049
Test Case Name	Clear Chat History
Endpoint	DELETE /chat/{trip_id}/clear
Preconditions	User is authenticated and owns the trip. Conversation has messages.
Input Data	Valid trip_id
Objective	Verify that clearing chat deletes all messages from the database and resets Redis conversation state.
Expected Result	HTTP 200 OK. All messages deleted, state_snapshot null, ai_session_id null.
Actual Result	Chat was cleared successfully: messages deleted, Redis state purged, snapshot reset.
Status	Passed

Table 5.2.50 Itinerary Card Data in Chat History Test Case

Test Case ID	TC-050
Test Case Name	Itinerary Card Data in Chat History
Endpoint	GET /chat/{trip_id}/history
Preconditions	Conversation has at least one AI message with itinerary card data.
Input Data	Valid trip_id with AI messages containing card_data
Objective	Verify that card_data (itinerary_data, hotel_options, booking_data, flight_options, rendered_cards) is persisted and returned correctly.
Expected Result	Messages with card_data include the full structured data for reconstructing UI cards.
Actual Result	Card data was correctly persisted and returned in history response.
Status	Passed

Table 5.2.51 State Recovery from DB Snapshot Test Case

Test Case ID	TC-051
Test Case Name	State Recovery from DB Snapshot
Endpoint	WS /ws/chat/{trip_id}
Preconditions	Redis is down or state expired. DB has state_snapshot with phase and slots.
Input Data	User sends a message after Redis state expired
Objective	Verify that the AI engine can recover structured state (phase, slots, turn_count) from the DB state_snapshot when Redis is unavailable.
Expected Result	The conversation resumes from the correct phase with slots intact.
Actual Result	State recovery worked: phase and slots were correctly restored from DB snapshot.
Status	Passed

Table 5.2.52 WebSocket Reconnection Preserves State Test Case

Test Case ID	TC-052
Test Case Name	WebSocket Reconnection Preserves State
Endpoint	WS /ws/chat/{trip_id}?token={token} (reconnect)
Preconditions	Trip has an ongoing conversation with existing itinerary. User disconnects and reconnects.
Input Data	Follow-up message after reconnection
Objective	Verify that reconnecting to the same trip WebSocket preserves conversation state (phase, history, ai_session_id).
Expected Result	AI engine recognizes the conversation, continues from the correct phase, and does not render duplicate itinerary cards.
Actual Result	Reconnection preserved state correctly and no duplicate cards were rendered.
Status	Passed

Table 5.2.53 Concurrent WebSocket Messages (Race Condition) Test Case

Test Case ID	TC-053
Test Case Name	Concurrent WebSocket Messages – Race Condition
Endpoint	WS /ws/chat/{trip_id}?token={token}
Preconditions	User is authenticated. Trip exists with an ongoing conversation. Backend server, Redis, and database are running.
Input Data	Two chat messages sent in rapid succession (< 50ms apart)
Objective	Verify that concurrent WebSocket messages are serialized to prevent race conditions — interleaved state mutations must not cause duplicate stops, lost profile updates, or phase corruption.
Expected Result	Messages are processed sequentially. No duplicate itinerary stops, lost profile updates, or inconsistent conversation phase transitions. Database state remains consistent.
Actual Result	Initial test showed interleaved state mutations — duplicate itinerary stops appeared and profile updates were lost when messages arrived within < 50ms of each other. After the fix, messages are properly serialized via per-user locking.
Status	Failed
Defect Found	Yes
Defect Description	- Two concurrent WebSocket frames could enter the AI engine's critical section at the same time, causing interleaved read-modify-write of the shared conversation state. - This resulted in duplicate itinerary stops, lost profile updates, and inconsistent conversation phase transitions.
Fix Applied	- Added a per-user lock (with a bounded cache of 1000 entries) so messages from the same user are processed one at a time. - Added a per-connection lock at the WebSocket route level as an extra safeguard. - Added Redis optimistic locking (WATCH/MULTI/EXEC) so conflicting saves are detected and retried instead of silently overwriting each other.

Category 12: Transparency & Continuous Improvement

Table 5.2.54 Submit Trip Feedback Test Case

Test Case ID	TC-054
Test Case Name	Submit Trip Feedback
Endpoint	POST /feedback/
Preconditions	User is authenticated. Trip exists and belongs to user.
Input Data	JSON with trip_id, feedback_type, rating, comment
Objective	Verify that a user can submit feedback (rating + comment) for a completed trip.
Expected Result	HTTP 201 Created with feedback_id, trip_id, rating, comment, submitted_at.
Actual Result	The request returned HTTP 201 with feedback created.
Status	Passed

Table 5.2.55 Get All Feedback by Current User Test Case

Test Case ID	TC-055
Test Case Name	Get All Feedback by Current User
Endpoint	GET /feedback/
Preconditions	User has submitted feedback for one or more trips.
Input Data	None
Objective	Verify that listing feedback returns all feedback entries by the authenticated user, ordered by most recent.
Expected Result	HTTP 200 OK with list of feedback entries ordered by submitted_at descending.
Actual Result	The request returned HTTP 200 with correct feedback list.
Status	Passed

Category 13: System Health & Monitoring

Table 5.2.56 Concurrent WebSocket Message Processing Test Case

Test Case ID	TC-056
Test Case Name	Concurrent WebSocket Message Processing
Endpoint	WS /ws/chat/{trip_id}?token={token}
Preconditions	Trip has an ongoing conversation.
Input Data	Two messages sent in rapid succession
Objective	Verify that concurrent messages are serialized using per-connection locks to prevent race conditions.
Expected Result	Messages are processed sequentially. No state corruption or duplicate data.
Actual Result	Concurrent messages were properly serialized.
Status	Passed

Table 5.2.57 WebSocket Chat with Invalid Token Test Case

Test Case ID	TC-057
Test Case Name	WebSocket Chat with Invalid Token
Endpoint	WS /ws/chat/{trip_id}?token={invalid_token}
Preconditions	Backend server is running.
Input Data	WebSocket connection with invalid/expired Firebase token
Objective	Verify that WebSocket connections with invalid tokens are rejected with close code 4001.
Expected Result	WebSocket close code 4001 (unauthorized).
Actual Result	The WebSocket connection was closed with code 4001.
Status	Passed

Chapter (6) Conclusions and Future Work

6.1 Conclusions

Throughout the development of the TourMate AI platform, the team successfully achieved several key milestones. The main conclusions are:

- **Agentic AI Pipeline:** designed a graph-based itinerary-generation pipeline in which six agents (Profile Loader, Place Retriever, Candidate Scorer, Planning Agent, Route Optimizer, and Validation Agent) collaborate within a shared state (TripState), complemented by a Preference Reranker and a Hotel Agent that the conversation orchestrator invokes outside the generation graph for preference shifts and post-approval accommodation selection.
- **Multimodal Preference Extraction:** used Google Gemini 2.5 Flash as a Vision-Language Model to analyze user-uploaded travel photos and extract style cues and preferences, enabling image-based personalization.
- **Unified Router Architecture:** replaced a fragmented three-call pattern with a single structured-output LLM call that handles intent classification, slot extraction, and response generation in one step, cutting latency by about 60% (see Chapter 5, Section 5.x).
- **Route Optimization:** used OSRM (Open Source Routing Machine) with a 2-opt local search algorithm to minimize travel time between stops, producing geographically coherent, time-efficient daily schedules.
- **Behavioral Profiling System:** built a per-trip profiling mechanism that learns from user actions, approvals, skips, and feedback to refine recommendations without needing explicit configuration.
- **Scalable Backend Architecture:** built the backend with asynchronous SQLAlchemy and asyncpg for non-blocking database access, Redis for session management with automatic expiry, multi-provider LLM configuration with key rotation, and LangSmith for tracing agent execution.
- **Cross-Platform Mobile Application:** created a mobile app using Flutter with a feature-first architecture, ensuring seamless access for users on both Android and iOS devices.
- **Comprehensive Testing:** conducted unit testing of the AI agents (pytest), integration testing of the full pipeline (FastAPI TestClient), and end-to-end testing of key user flows; all 12 documented test cases passed, confirming the system's reliability.

Testing confirmed that TourMate AI meets its functional requirements, including user authentication, trip management, AI-powered itinerary generation, place discovery, flight and hotel booking simulation, payment processing, and review management. Overall, TourMate AI bridges the gap between manual travel planning and static automated tools,

showing how agentic AI can handle complex, real-world planning problems while delivering adaptive, personalized itineraries.

6.2 Lessons Learned

Through the journey of this project, the team gained valuable lessons in both technical and project management aspects:

Technical Lessons

- **Multi-Agent System Design:** using LangGraph showed the importance of clear state management and conditional routing. The shared TripState had to be carefully designed so each agent could access only the data it needed, and adding automatic retry loops to the Validation Agent made the pipeline noticeably more robust.
- **Prompt Engineering for LLMs and VLMs:** writing effective prompts for Gemini 2.5 Flash and Groq Llama models took several iterations. For vision tasks, giving the Vision-Language Model explicit categories and examples produced more consistent results than open-ended prompts.
- **Integrating New Technologies Requires Deep Research:** working with structured-output LLMs, vision models, and route optimization highlighted the importance of thorough research and testing to ensure performance and accuracy.
- **Managing AI Non-Determinism:** large language models are inherently non-deterministic. This was addressed with structured output schemas (Pydantic models) to enforce valid JSON, combined with a Validation Agent that checked feasibility and quality, triggering retries when validation failed.
- **Service Integration Strategy:** using a provider-agnostic design from the start made it easy to swap or upgrade external services (Firebase, OSRM, LangSmith) without restructuring the system, and a multi-key rotation scheme kept the platform reliable within free-tier LLM limits.
- **Database and Backend Design:** managing 30 Alembic migrations highlighted the value of careful schema design and small, incremental changes, while Redis session management with a one-hour expiry worked well for preserving state across WebSocket reconnections.

Project Management Lessons

- **Agile Flexibility is Crucial:** adopting a hybrid Waterfall-Agile approach allowed the team to continuously adapt to challenges and validate each subsystem before integration, reducing the risk of compounding failures.
- **Team Collaboration and Task Distribution:** effective coordination and clear role assignments were essential in managing the complex and parallel development tracks of the system (backend, AI agents, and mobile app).

- **Testing Early and Often Matters:** treating testing as an ongoing activity, not a final step, helped identify issues before they escalated, leading to smoother integrations and a more reliable platform.

6.3 Future Work

To further enhance the TourMate AI platform and expand its capabilities, several key areas have been identified for future development. These initiatives aim to improve user experience, streamline operations, broaden market reach, and integrate more advanced technologies.

- **Real-World Booking Integration:** connect to real booking APIs, such as Booking.com, Expedia, or Agoda, for accommodations, and upgrade to the Amadeus production API for live flight booking and ticketing, enabling real-time pricing, availability, and confirmed reservations.
- **Global and Language Expansion:** expand the place database beyond Egyptian destinations (currently Cairo, Alexandria, Luxor, and Aswan) to major international cities, paired with Arabic-language support followed by other languages such as French, German, and Mandarin.
- **Real Payment Gateway Integration:** integrate with local payment gateways such as Fawry and PayMob to offer secure, diverse, and convenient payment options for customers during booking.
- **Push Notifications and Smart Reminders:** integrate Firebase Cloud Messaging to enable real-time push notifications for trip milestones, such as upcoming flights, check-in reminders, and weather alerts.
- **Review Sentiment Analysis:** use natural language processing (NLP) to perform sentiment analysis on customer reviews and aggregate ratings from platforms like Google Maps and TripAdvisor, enriching the place database with deeper insight.
- **Multi-Modal Transportation Optimization:** extend route optimization beyond walking and driving to include public transit schedules, ride-sharing, and inter-city transport such as trains, buses, and domestic flights.
- **Offline Itinerary Access:** implement offline caching of itineraries, saved places, and trip details, allowing travelers to access their plans without an internet connection.
- **Collaborative Trip Planning:** add support for multi-user trip planning, allowing groups of travelers to jointly build, discuss, and vote on itinerary options.

References

- [1] S. Sureddi, "Adopting Agile Methodologies and Frameworks in Automotive Industry." [Online]. Available: <https://urfjournals.org/open-access/adopting-agile-methodologies-and-frameworks-in-automotive-industry.pdf>. Accessed: Nov. 2025.
- [2] Vacay, "AI Travel Assistant Overview." [Online]. Available: <https://www.usevacay.com/>. Accessed: Dec. 2025.
- [3] Wanderlog, "Free Travel Planner & Itinerary App." [Online]. Available: <https://wanderlog.com/>. Accessed: Dec. 2025.
- [4] Roam Around, "AI Travel Assistant." [Online]. Available: <https://roamaround.app/>. Accessed: Dec. 2025.
- [5] Google Travel, "Plan Your Trip with Google." [Online]. Available: <https://www.google.com/travel/>. Accessed: Dec. 2025.
- [6] GuideGeek, "Your Personal AI Travel Assistant." [Online]. Available: <https://guidegeek.com/>. Accessed: Dec. 2025.
- [7] FastAPI, "FastAPI Documentation." [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: Jan. 2026.
- [8] Flutter, "Flutter Documentation." [Online]. Available: <https://docs.flutter.dev/>. Accessed: Sep. 2025.
- [9] LangChain, "LangGraph Overview." [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: Oct. 2025.
- [10] Google AI for Developers, "Gemini 2.5 Flash." [Online]. Available: <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>. Accessed: Feb. 2026.
- [11] Groq, "GroqCloud API Reference." [Online]. Available: <https://console.groq.com/docs/api-reference>. Accessed: Feb. 2026.
- [12] Meta AI, "Llama 3.3-70B-Instruct Model Card." [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>. Accessed: Feb. 2026.

- [13] G. A. Croes, "A Method for Solving Traveling-Salesman Problems," *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958. [Online]. Available: <https://pubsonline.informs.org/doi/10.1287/opre.6.6.791>. Accessed: Mar. 2026.
- [14] The PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: <https://www.postgresql.org/docs/>. Accessed: Sep. 2025.
- [15] Redis, "Redis Documentation." [Online]. Available: <https://redis.io/docs/latest/>. Accessed: Mar. 2026.
- [16] Google, "Firebase Authentication Documentation." [Online]. Available: <https://firebase.google.com/docs/auth>. Accessed: Mar. 2026.
- [17] Project OSRM, "Open Source Routing Machine." [Online]. Available: <https://project-osrm.org/>. Accessed: Mar. 2026.
- [18] Amadeus for Developers, "Self-Service APIs Documentation." [Online]. Available: <https://developers.amadeus.com/self-service/apis-docs>. Accessed: Apr. 2026.
- [19] Stripe, "Stripe API Reference." [Online]. Available: <https://docs.stripe.com/api>. Accessed: May. 2026.
- [20] LangChain, "LangSmith Observability." [Online]. Available: <https://docs.langchain.com/langsmith/observability>. Accessed: Mar. 2026.